



第七章 密文查询

目 录

「01」

知识点五：保留顺序加密

「02」

知识点六：顺序揭示加密

「03」

知识点七：频率隐藏OPE

「04」

知识点八：FH-OPE实践

「05」

知识点九：密态数据库基础

「06」

知识点十：CryptDB数据库

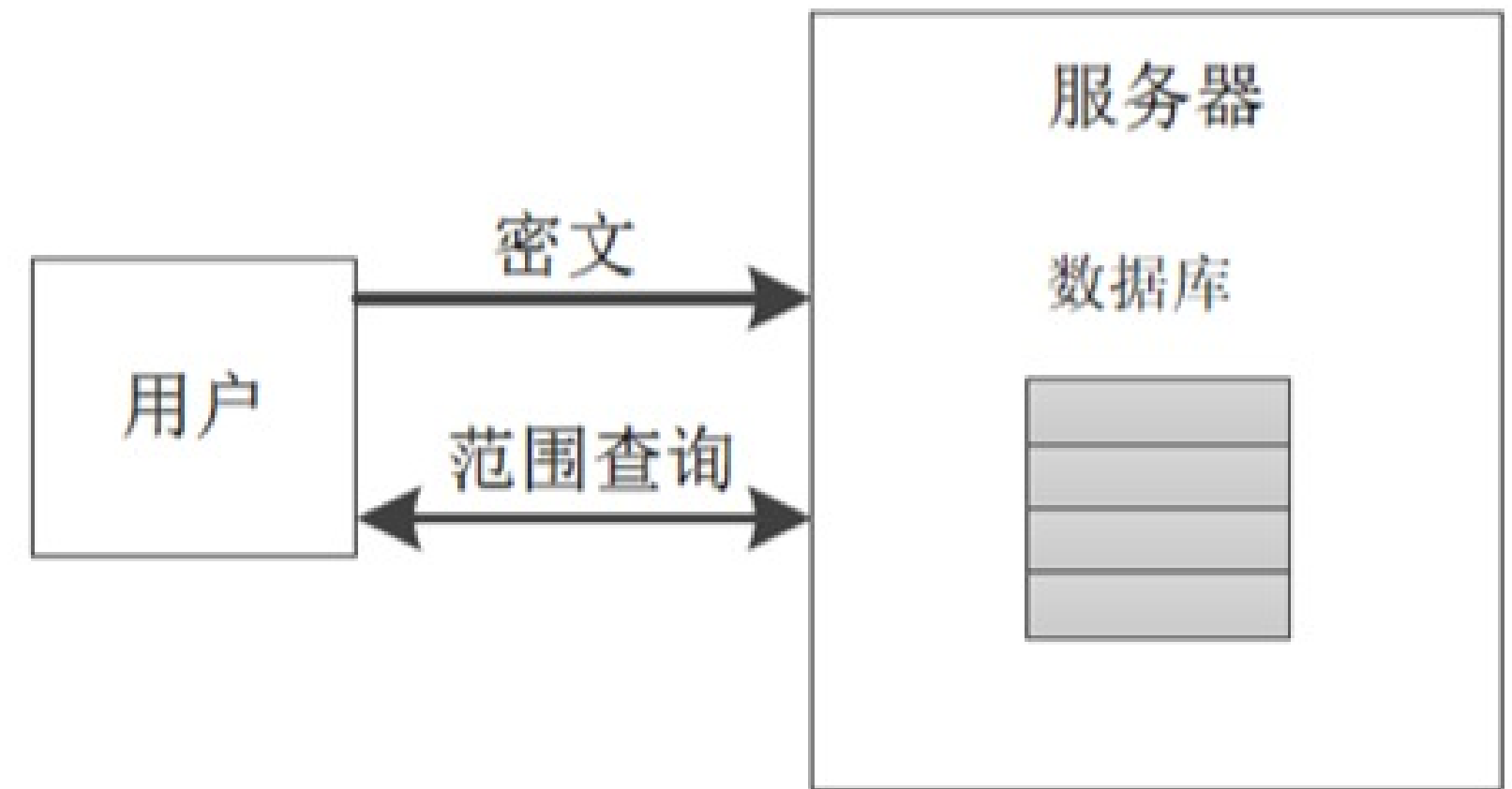
1. 保留顺序加密

○ 保留顺序加密 (Order Preserving Encryption)

指明文的顺序和密文的顺序是匹配的，即如果明文a和b满足 $a < b$ ，那么经过加密后的密文 $Enc(a)$ 和 $Enc(b)$ 也满足 $Enc(a) < Enc(b)$ 。

举例说明： 明文： 1 4 4 5 5 7

密文： 13 16 16 19 19 21



○ 在数据库中的应用

□ 密文有序，无需额外操作，可以直接支持密文上的Max、Min、Order by等SQL操作

□ Select * from T where **C**>5 and **C**<10 → Select * from T where **C**>Enc(5) and **C**<Enc(10)

1. 保留顺序加密

○ 理想安全性: IND-OCPA (indistinguishability under **ordered chosen-plaintext attack**)

只允许泄露密文的顺序、不允许泄露其它信息

攻击者在执行一定次数查询后, 提交两个有顺序的明文:

猜测返回的密文是哪个明文的密文的概率是可忽略的

○ 举例: 1 2 3 4 5 6 7 8 9 10

○ 典型OPE方案: BCLO (Boldyreva 等人, 2009)

利用**超几何分布**使得产生的密文距离变的随机, 密文扩充

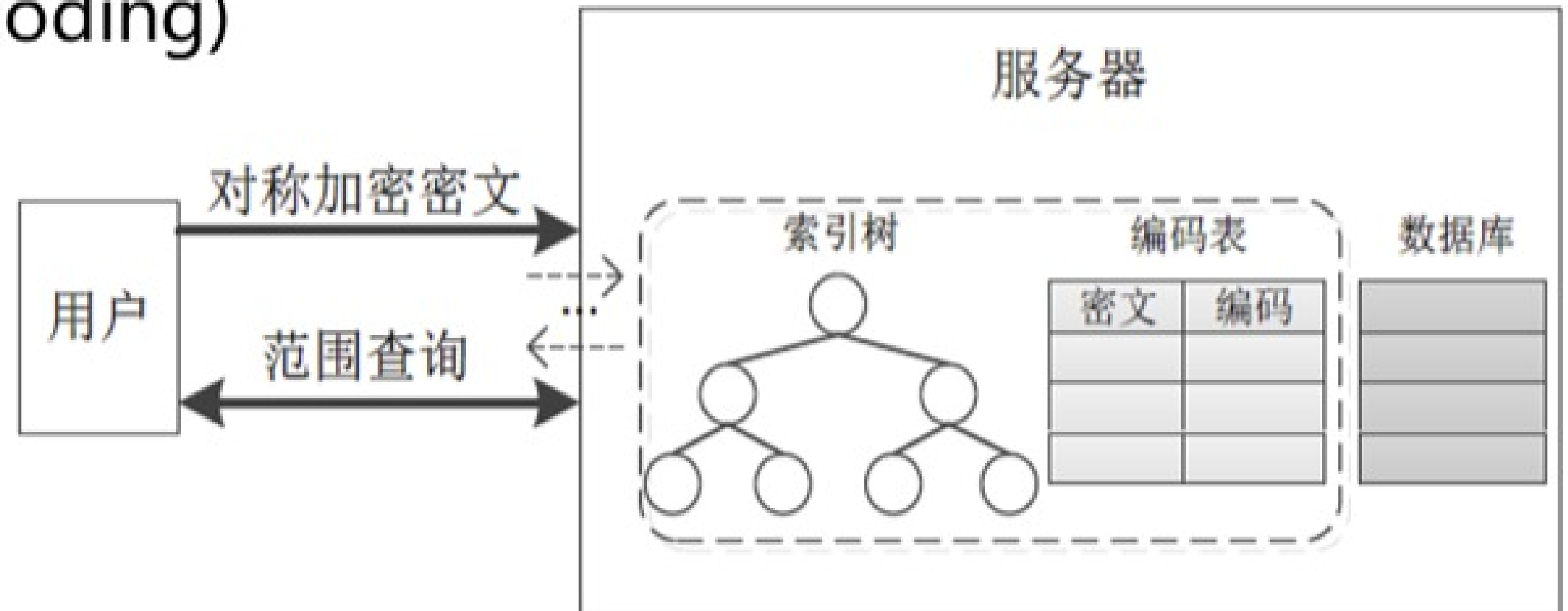
达到理想IND-OCPA: 要求**指数级**密文长度, 让密文距离差异变的可忽略, 不可推测

存在泄露: 任意**明文近似值**以及任意两个明文之间的**近似距离**, 精度约为域大小的平方根 ; 当密文域是明文域的平方倍和立方倍的时候, 该方案泄露高位的一半明文比特

2. 顺序保留编码

○ 顺序保留编码 (Order Preserving Encoding)

- ✓ 密文不要求有顺序和可比较
- ✓ 服务器端维护密文和其对应的编码
- ✓ 服务端通常还维护额外的索引树



○ 优点： 通常具有较理想的安全性，可以达到IND-OCPA安全性

○ 缺点： 客户端与服务器端存在交互

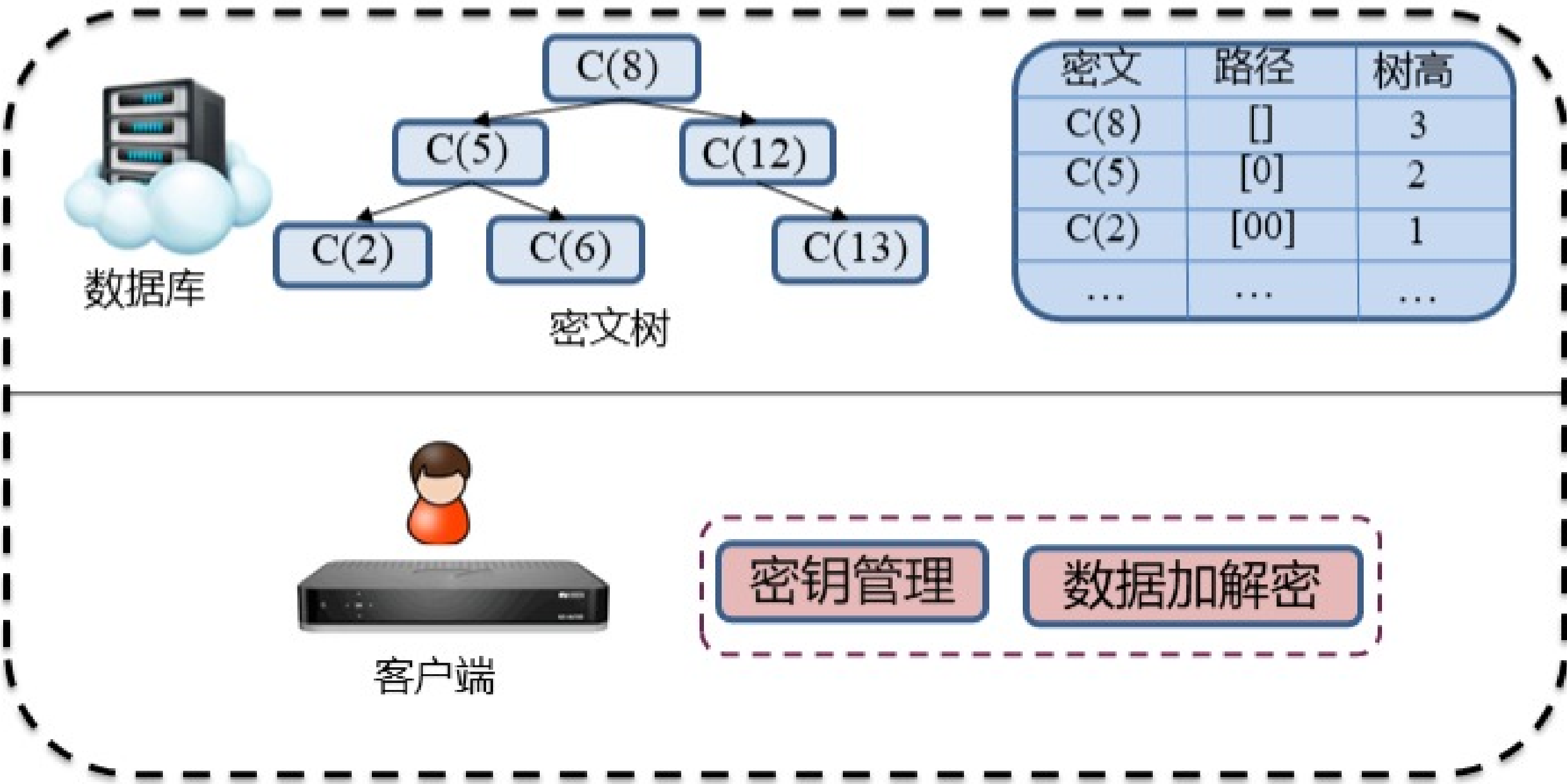
- ✓ 插入时：多次交互遍历索引树生成密文编码，更新编码表并将编码插入数据库
- ✓ 查询时：基于编码表得到密文对应的编码，然后再进行范围查询

2. 顺序保留编码

○ 经典顺序保留编码方案: mOPE (Popa et al. 2013)

服务器端维护排序树:

节点值为数据的对称加密密文
左子树中的密文小于右子树密文



操作有交互: 确定插入位置 | 查询可以通过索引树确定范围 | 树存在平衡的调整且需交互

目 录

「01」

知识点五：保留顺序加密

「02」

知识点六：顺序揭示加密

「03」

知识点七：频率隐藏OPE

「04」

知识点八：FH-OPE实践

「05」

知识点九：密态数据库基础

「06」

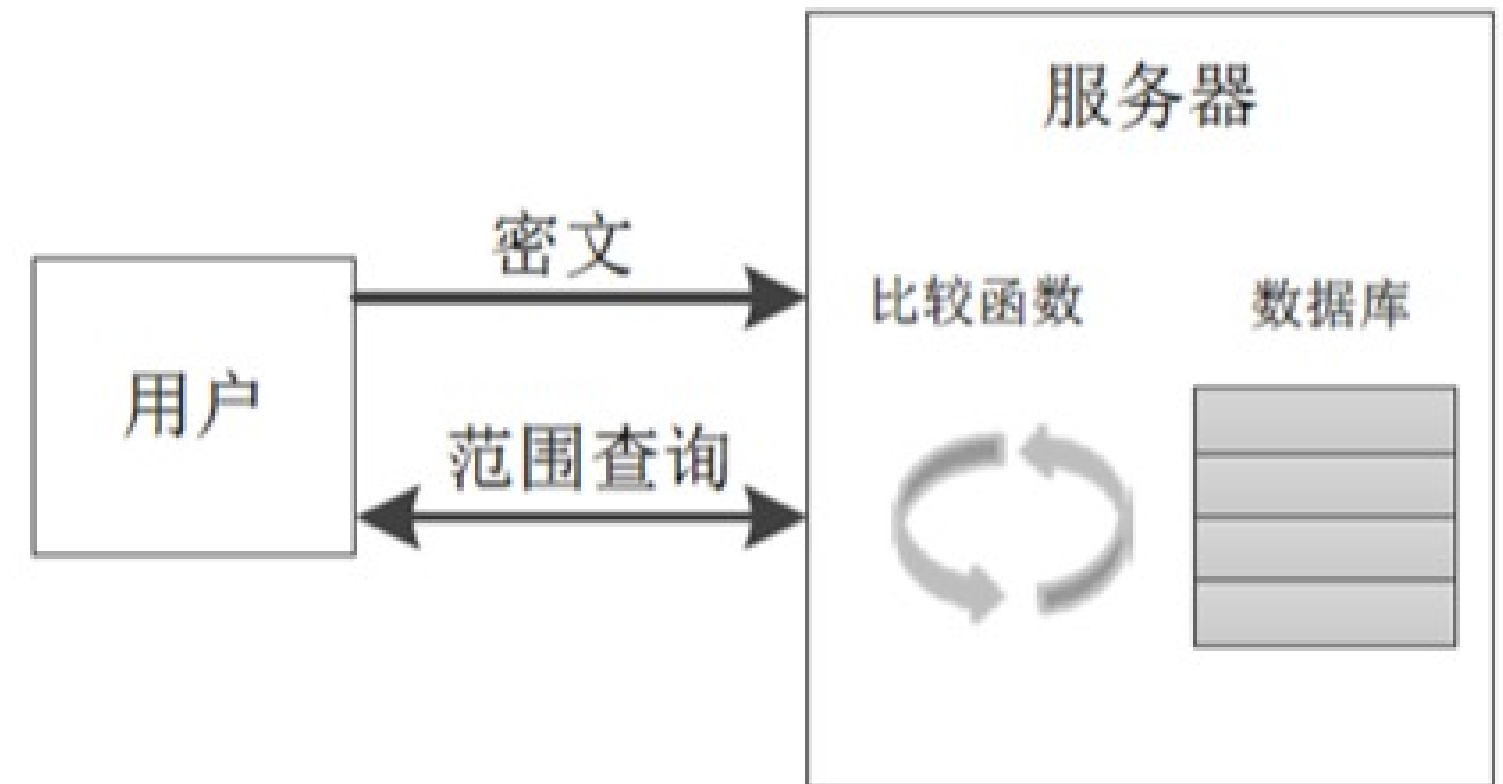
知识点十：CryptDB数据库

1. 顺序揭示加密

○ 顺序揭示加密 (Order Revealing Encryption)

定义：密文不直接泄露顺序，允许通过一个比较函数计算密文的顺序。

理想安全性：比较的时候除了顺序，其它不泄露



○ 数据库应用模式一：直接存储ORE密文

缺点：范围查询需要对所有密文进行线性扫描

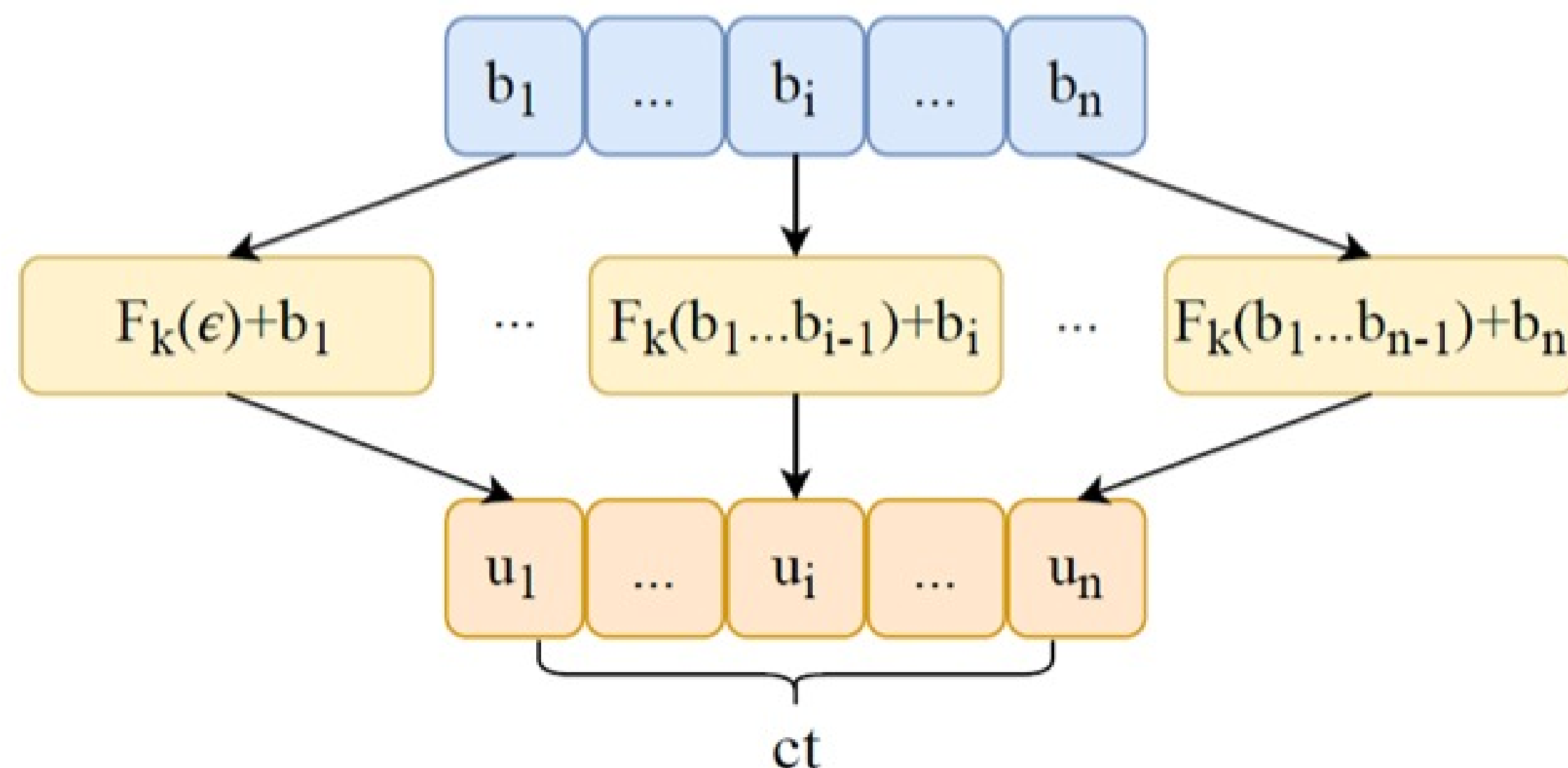
○ 数据库应用模式二：融合保留顺序编码用于构建密文索引树

思路：类似mOPE用于建立索引树，通过ORE来替代客户端与服务器端的交互

挑战：数值比较的时候不能泄露除了顺序之外的信息（理想IND-OCPA的方案才可用）

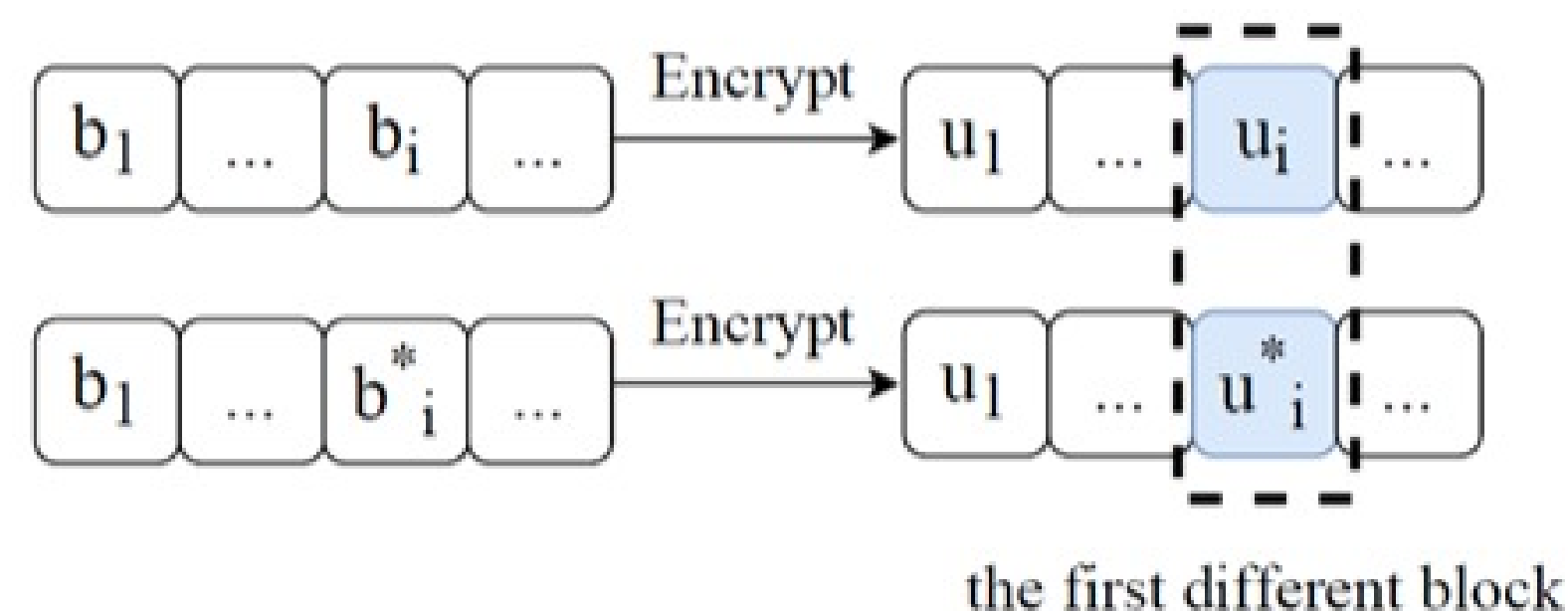
2. 典型ORE方案

○ 典型方案一：CLWW (Chenette et al. 2016)



$$u_i = F(k, (i, b_1 b_2 \dots b_{i-1} \| 0^{n-i})) + b_i \pmod{\mathbf{M}},$$

$\mathbf{M} = 2d - 1$ d 是进制 -- 此模运算可保留大小



- 优点: (1) 密文可直接比较、无需陷门
(2) 高效、逐位比较

- 缺点: 泄露第一个不同的bit (密文)
→ 两个明文之间的距离

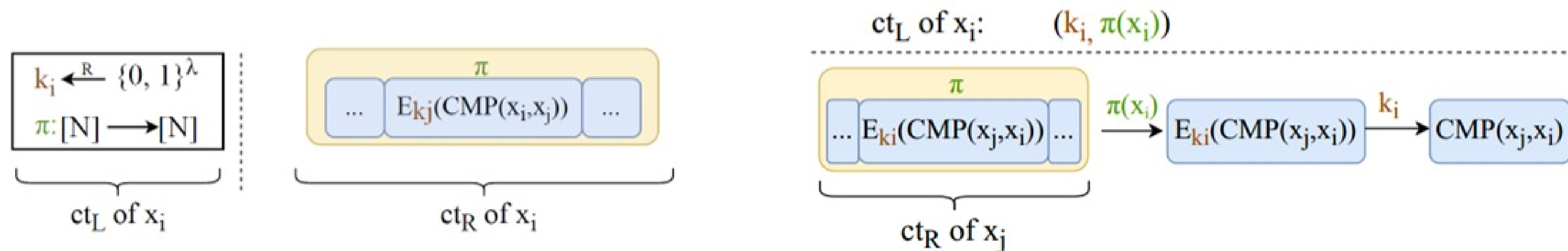
2. 典型ORE方案

○ 典型方案二：Lewi-WU Small (Lewi et al. 2016)

适合小域： $x_1, x_2, \dots, x_i, \dots, x_n$

每个元素：左右两半密文，左密文索引位置(客户端)、右密文存储和其它元素的比较结果(服务器端)

元素比较： x_i 的左半部分(作为陷门提交) & x_j 的右半部分(服务器端存储的密文)



□ 优点：理想安全 $\leftrightarrow$ ct_L 存储在客户端或经由客户端计算

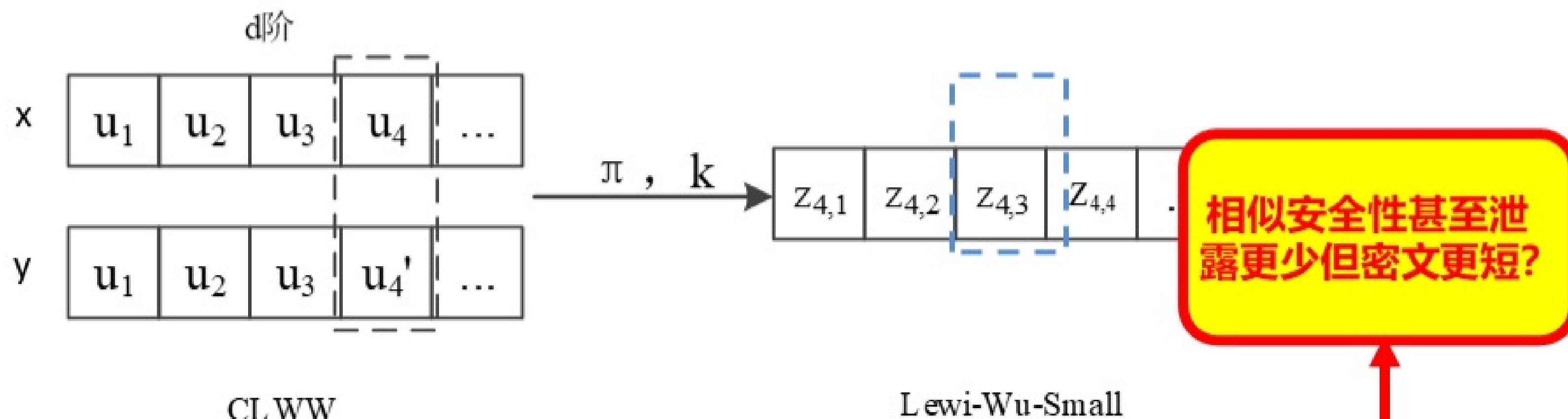
□ 缺点：密文扩充（与小域内元素成线性）、需要提交比较陷门

2. 典型ORE方案

○ 典型方案二：Lewi-WU Normal (Lewi et al. 2016)

分块：适合大域，对二进制位串根据d阶分块，分成n个块 $\leftrightarrow$ 将位的比较变为块的比较

方案：块块比较-CLWW，块内比较Lewi-Wu-Small



□ **密文扩充：**可以工作在大域，但仍然存在密文扩充，与d阶小域内元素个数呈现线性关系

□ **仍有泄露：**达不到理想安全性，将泄露第一个不相等的块 $\leftrightarrow$ 折衷的安全性 $>$ CLWW

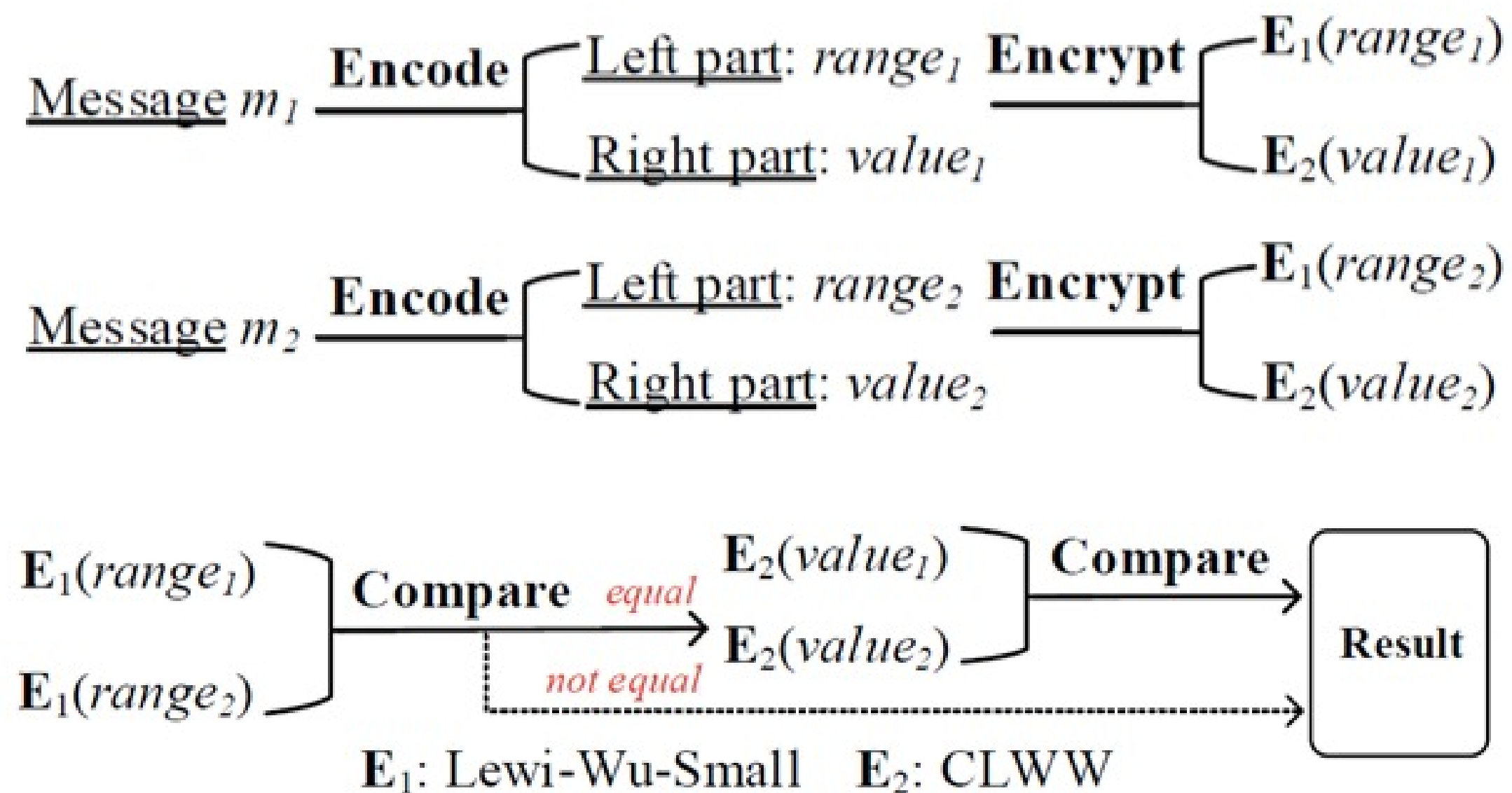
3. 混合模型

- 混合模型：同时提升安全性和效率

将明文划分为两部分：范围 | 数值

范围比较：理想安全小域ORE、不泄露信息

数值比较：如果范围相同，则比较数值，数值二进制位左侧对齐，逐位比较CLWW



挑战：

如何表示范围和数值？ $\leftrightarrow$ 编码策略

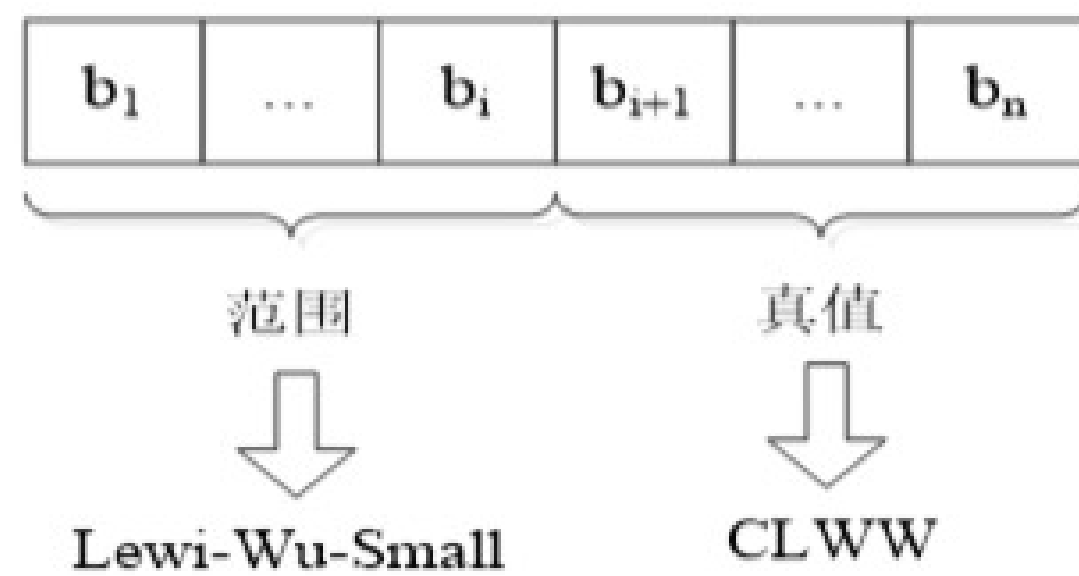
如何让范围比较和数值比较关联起来？

3. 混合模型

编码策略：类科学计数法 $\text{num} = \mathbf{V} * d^{\mathbf{E}}$ 指数E是范围、小数V是数值

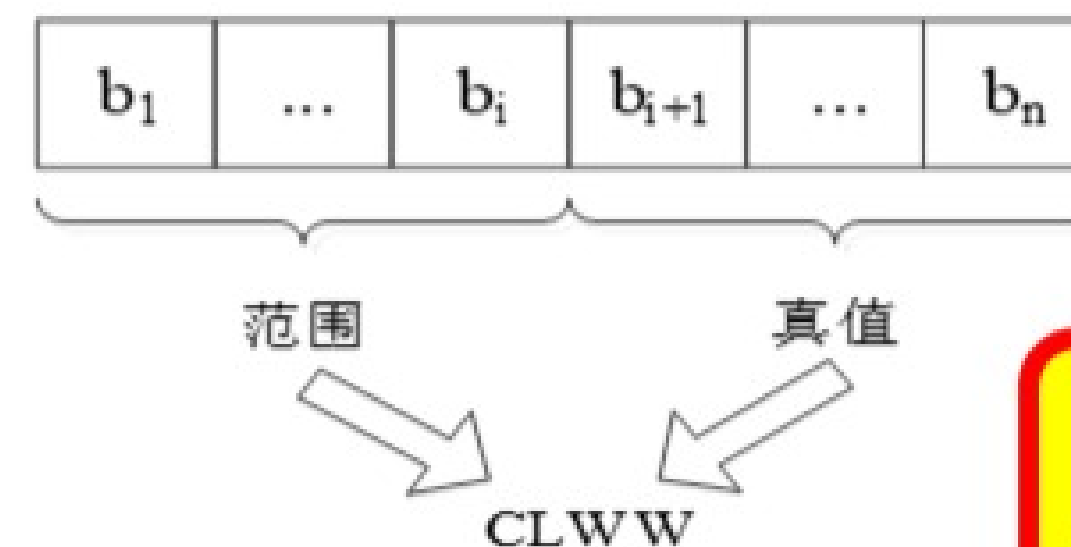
E || V 的二进制编码不会破坏比较操作（数值或字符串均适用）

○ 方案一：Hybrid ORE



- 分立比较：如何关联？交互？
- 信息泄露：泄露小数首个不等位
- 密文较长：小域仍然线性扩展

○ 方案二：Encode ORE



- 缩短密文：采用CLWW
- 建立关联：逐位加密和比较
- 信息泄露：范围相同-泄露小数首个不等位
范围不同-泄露第一个不相等块

安全性
 $\geq$ Lewi-wu-Normal

4. 实验结果

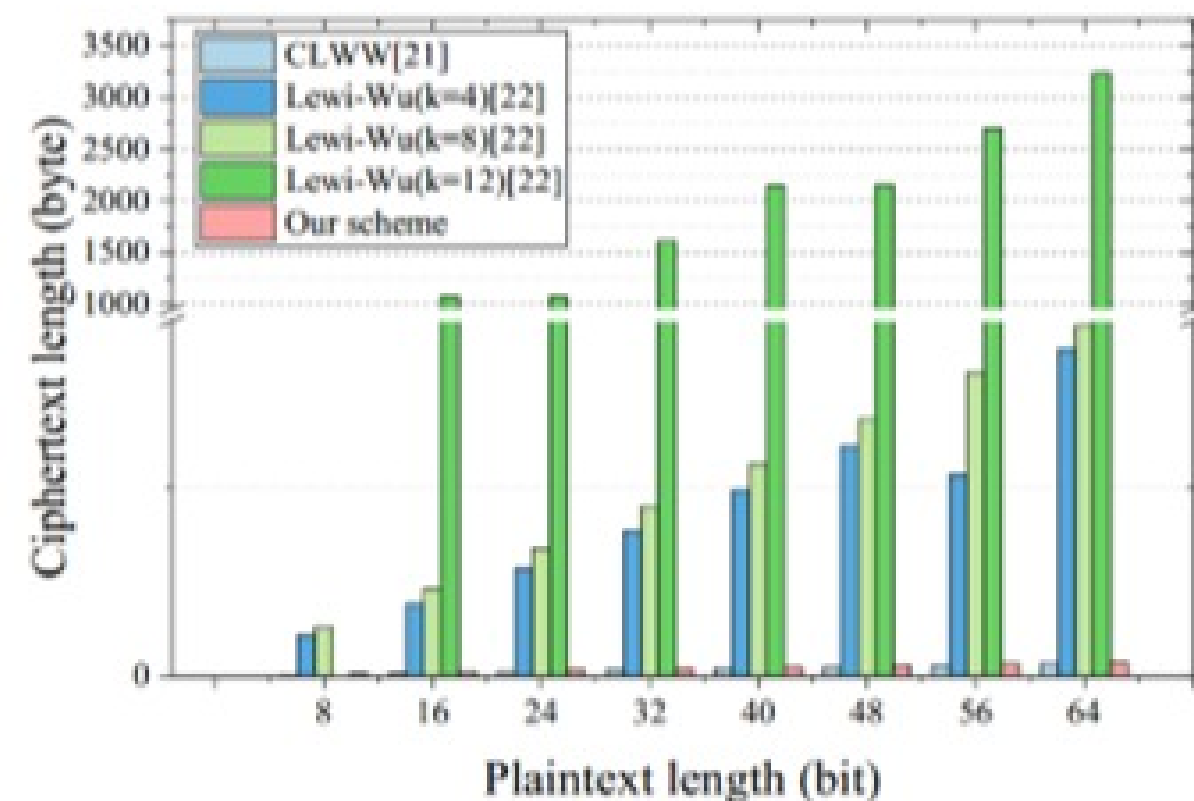


Fig. 8: Comparison of ciphertext length with different plaintext length.

密文长度的比较

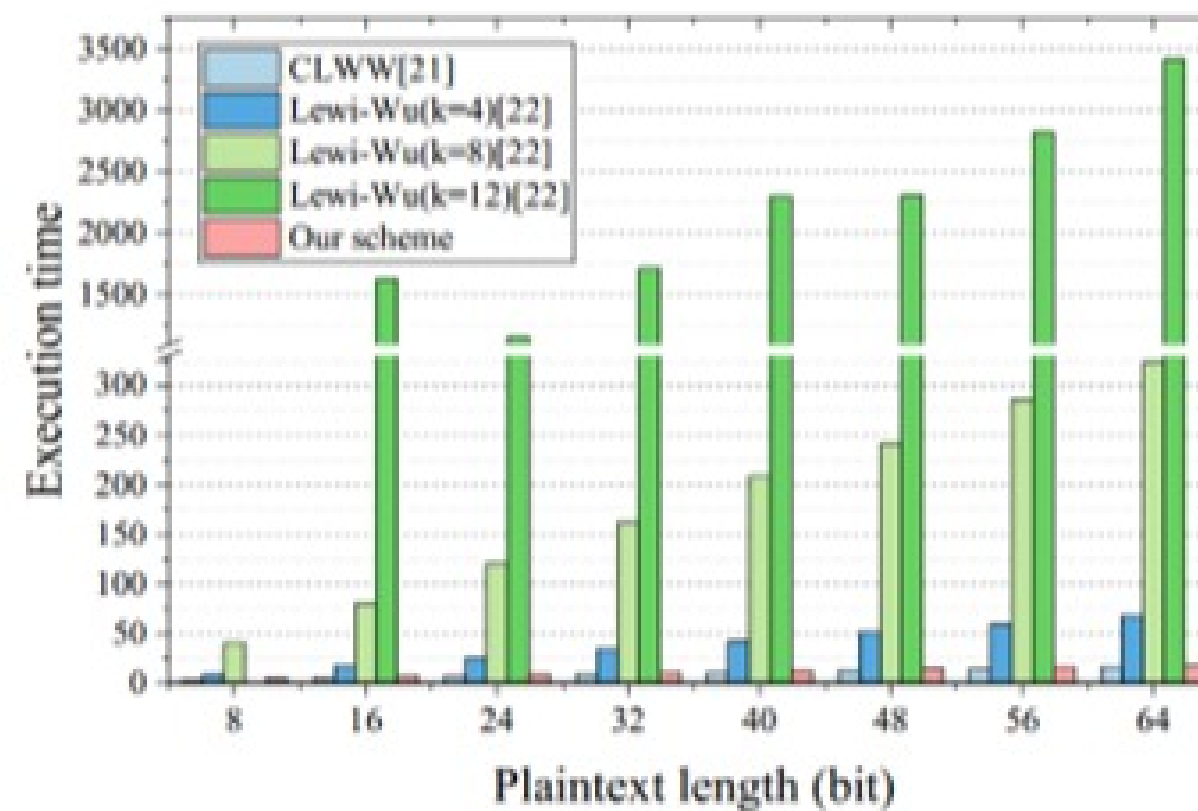


Fig. 11: Overall execution time with different plaintext length.

执行时间（加密，比较）的比较

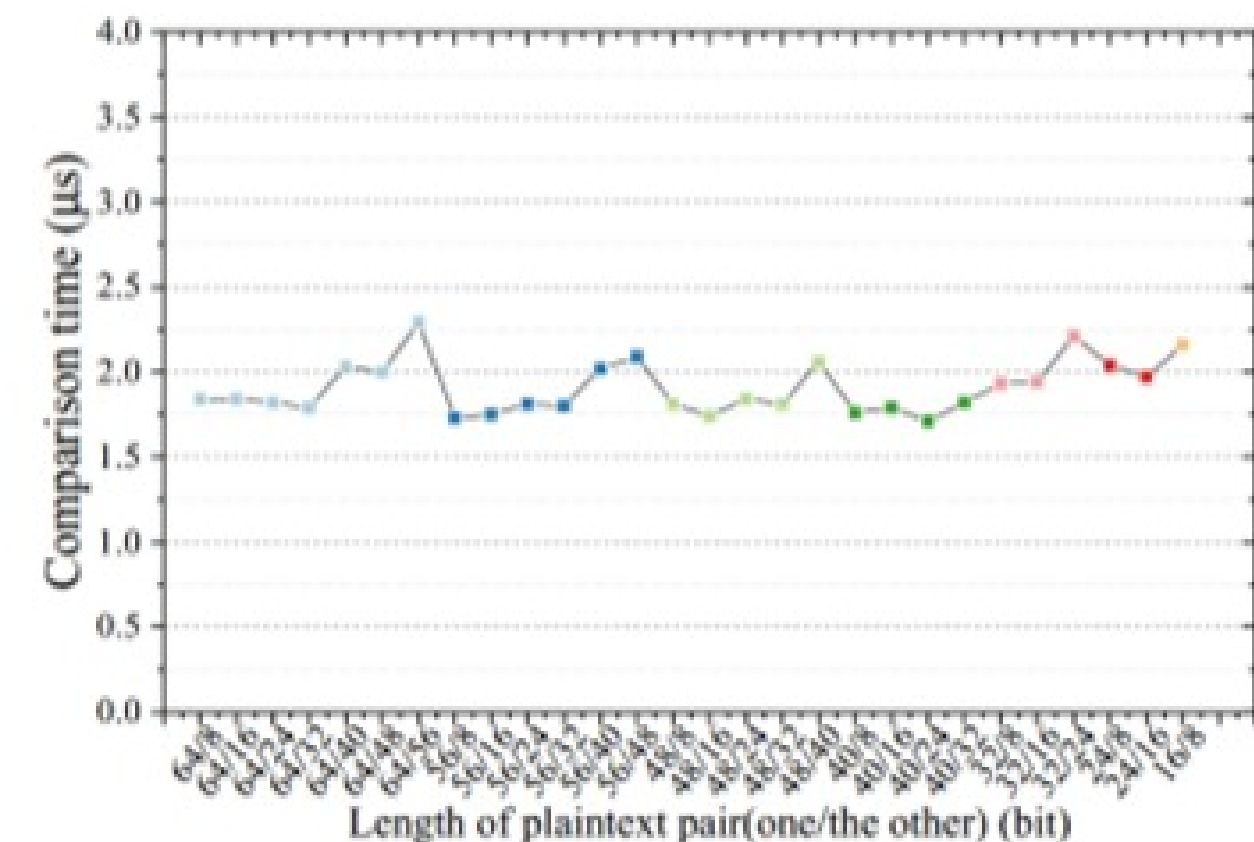


Fig. 12: Comparison time for plaintext over different lengths.

明文长度不同时，EncodeORE比较操作的时间

目 录

「01」

知识点五：保留顺序加密

「02」

知识点六：顺序揭示加密

「03」

知识点七：频率隐藏OPE

「04」

知识点八：FH-OPE实践

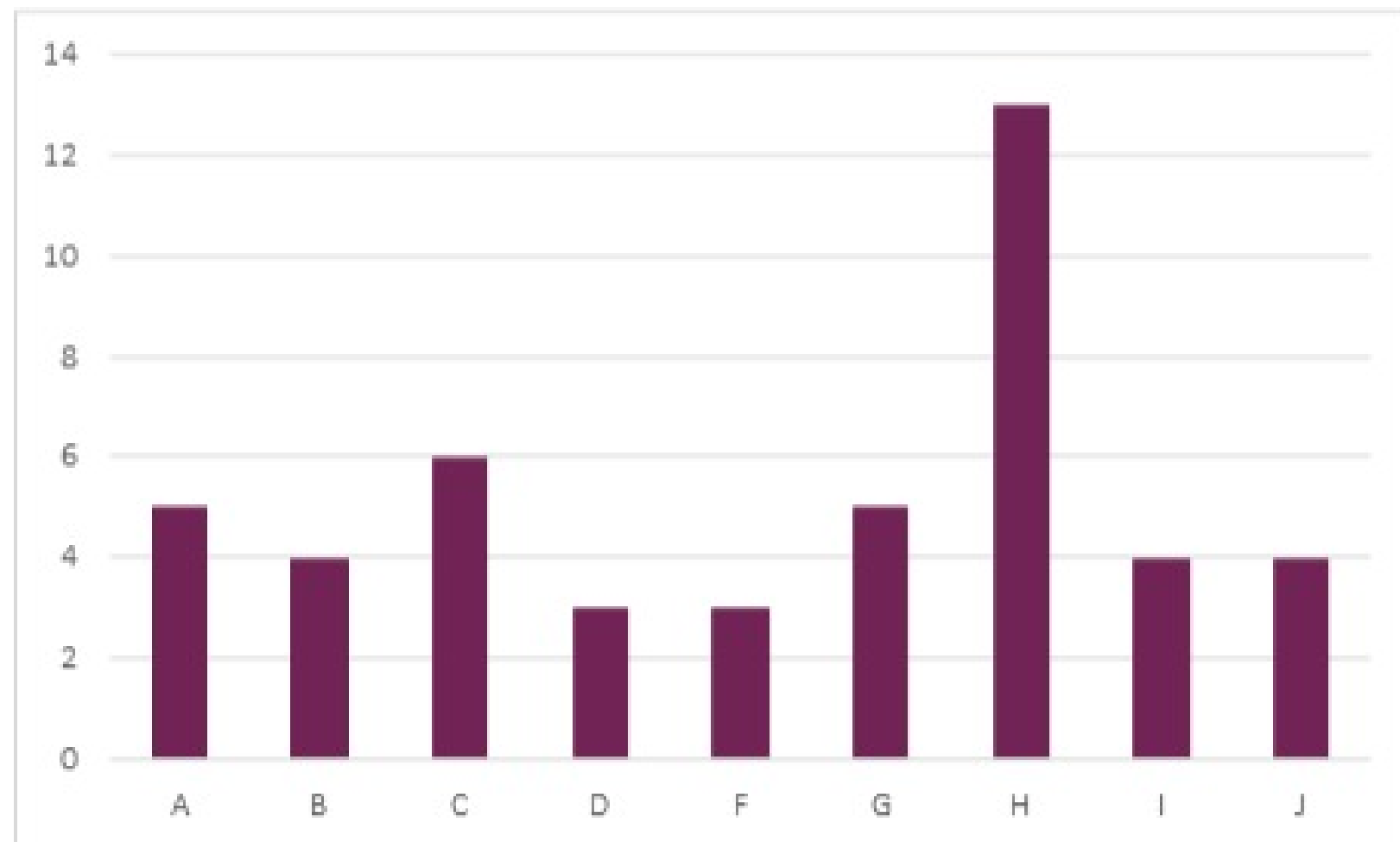
「05」

知识点九：密态数据库基础

「06」

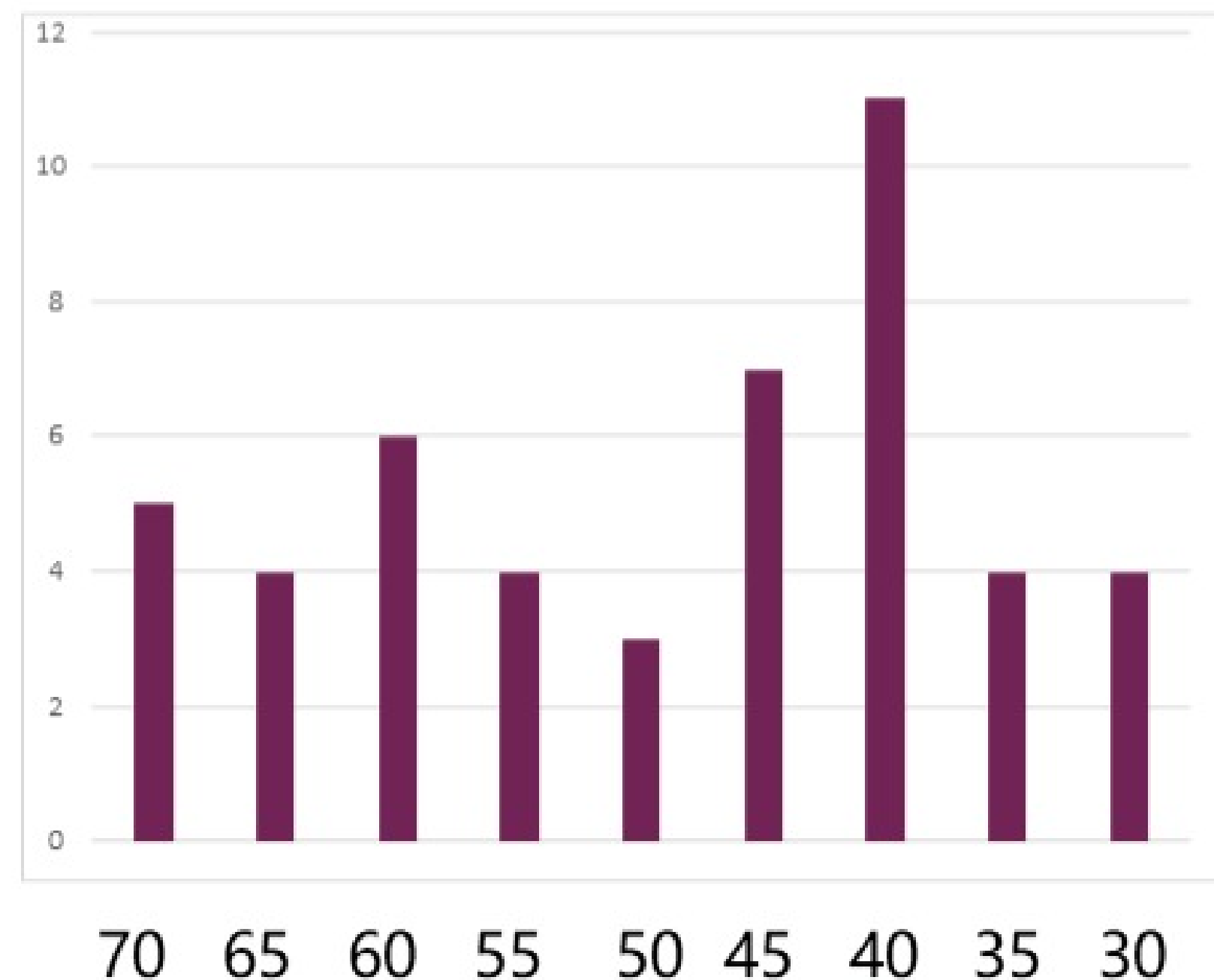
知识点十：CryptDB数据库

1. 频率推理攻击



南开大学网安学院2021年不同年龄段的教师数量

假定OPE方案已经达到了IND-OCPA安全
是不是就可以安全使用了？



辅助信息

南开大学网安学院2020年不同年龄段的教师数量

2.频率隐藏保序加密方案

与确定性保序加密不同：密文 $\text{Enc}(a)$ 和 $\text{Enc}(b)$ 如果满足 $\text{Enc}(a) < \text{Enc}(b)$ 则 $a \leq b$ 。

(相同明文的密文顺序任意，不同明文的密文顺序保留)

○ 举例说明

明文： 1 4 4 5 5 7

密文： 13 14 16 17 19 21

或密文： 13 16 14 19 20 21

相同的明文加密成不同的密文，且
不应依赖于加密顺序，是随机的

2.频率隐藏保序加密方案

○ FH-OPE: 无交互的纯客户端的频率隐藏OPE

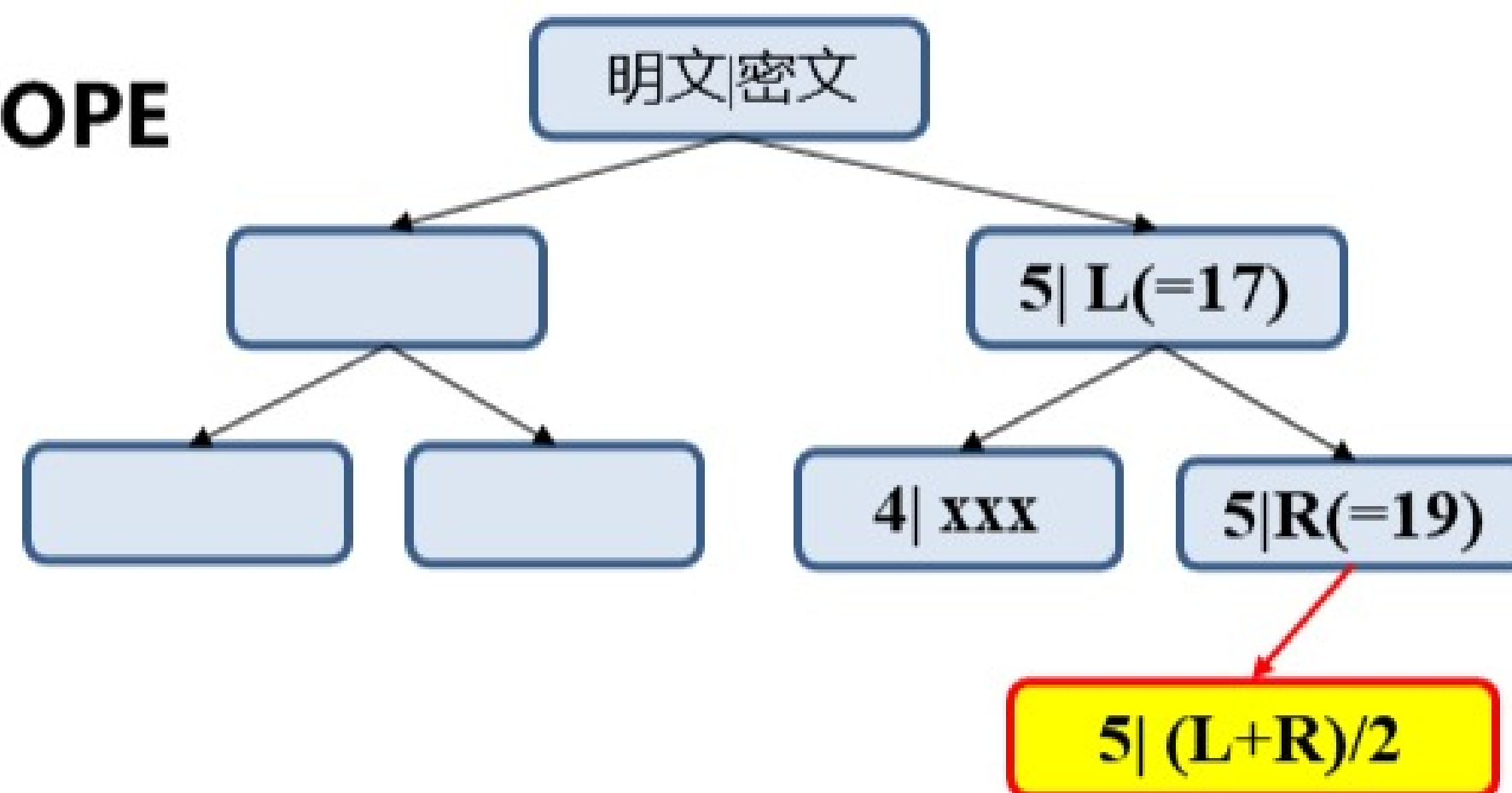
客户端维护一个排序树

节点存储: 明文值|密文值

插入一个新节点的时候:

位置: 寻找一个相应随机的插入位置, 以5为例, 可4右侧或第二个5的左右侧

密文值计算: $(L+R)/2$



□ **客户端存储大:** 存储全部明文及对应的密文

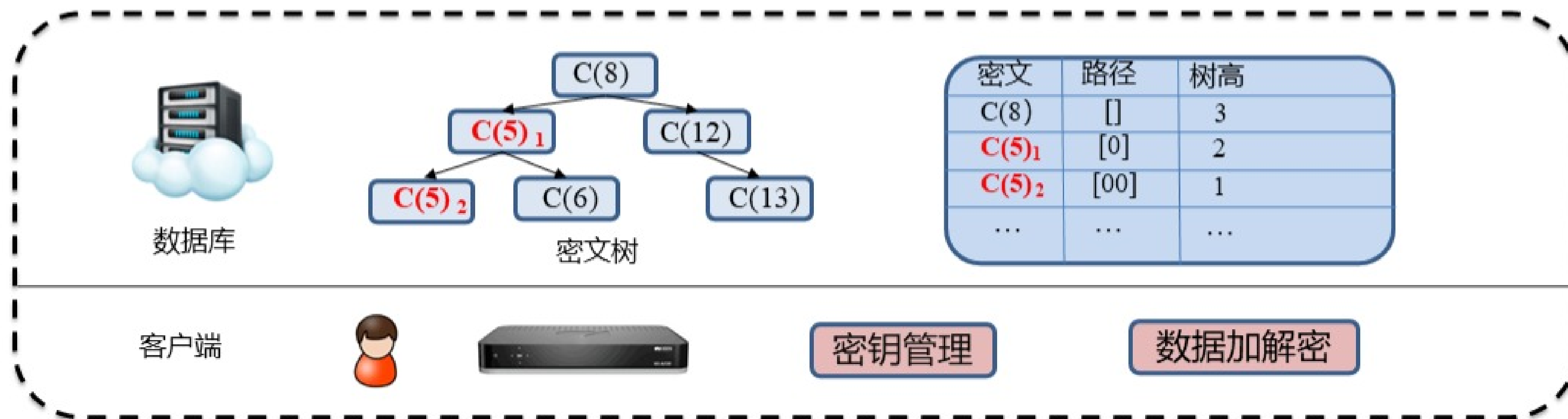
□ **更新数据量大:** 排序树平衡调整或无值可产生会更新

一旦更新, 将对全树的密文值进行更新

2.频率隐藏保序加密方案

降低客户端存储的思路：在服务器端保存状态

低存储客户端是数据库应用基本要求

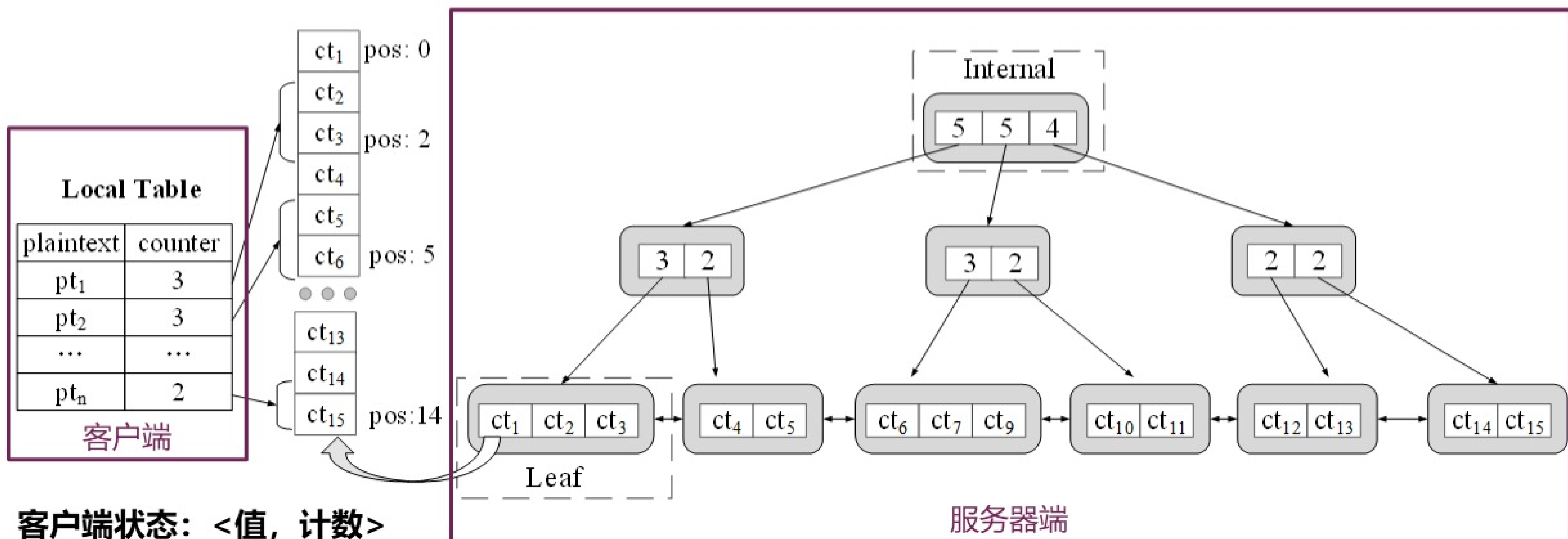


□ **做法:** 相同值随机选择一个相等节点的左右两侧插入

□ **缺点:** 客户端和服务端交互太多、排序树不平衡引发的树调整和密文编码更新太多

3.基于编码树的FH-OPE

如何进一步降低交互和密文更新？ 客户端-服务器协同 → 目标：无额外交互



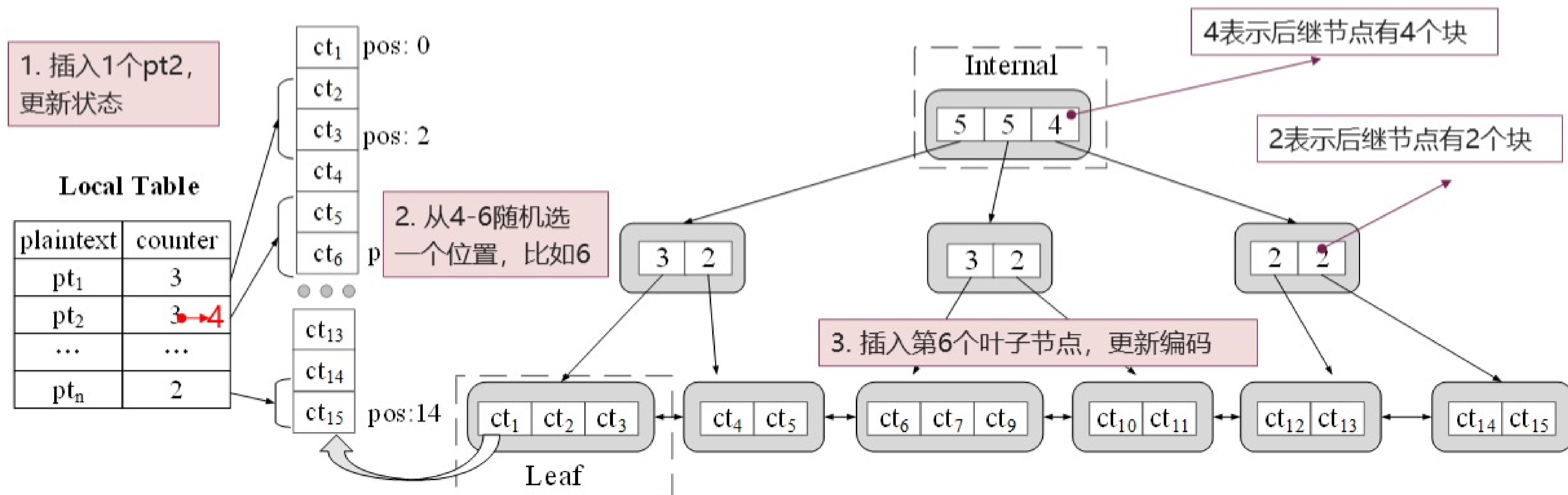
客户端状态：<值，计数>

□ 目的是计算插入位置

服务器状态：编码树(类B+树结构)，目的是减少树调整带来的密文更新

3.基于编码树的FH-OPE

分支节点结构：最多m个后继节点，块关键字为后继节点存储的块数量($\leq m$)



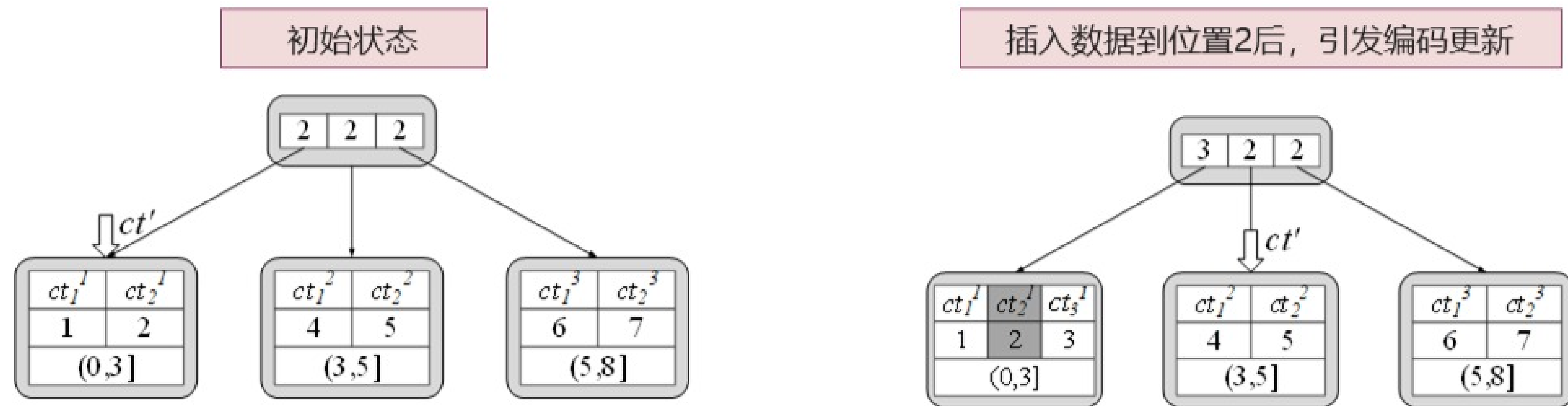
插入一个节点：(1)客户端更新counter并计算插入位置; (2)服务器端根据位置插入

3.基于编码树的FH-OPE

叶子节点： 存储密文数据以及所对应的一个编码值

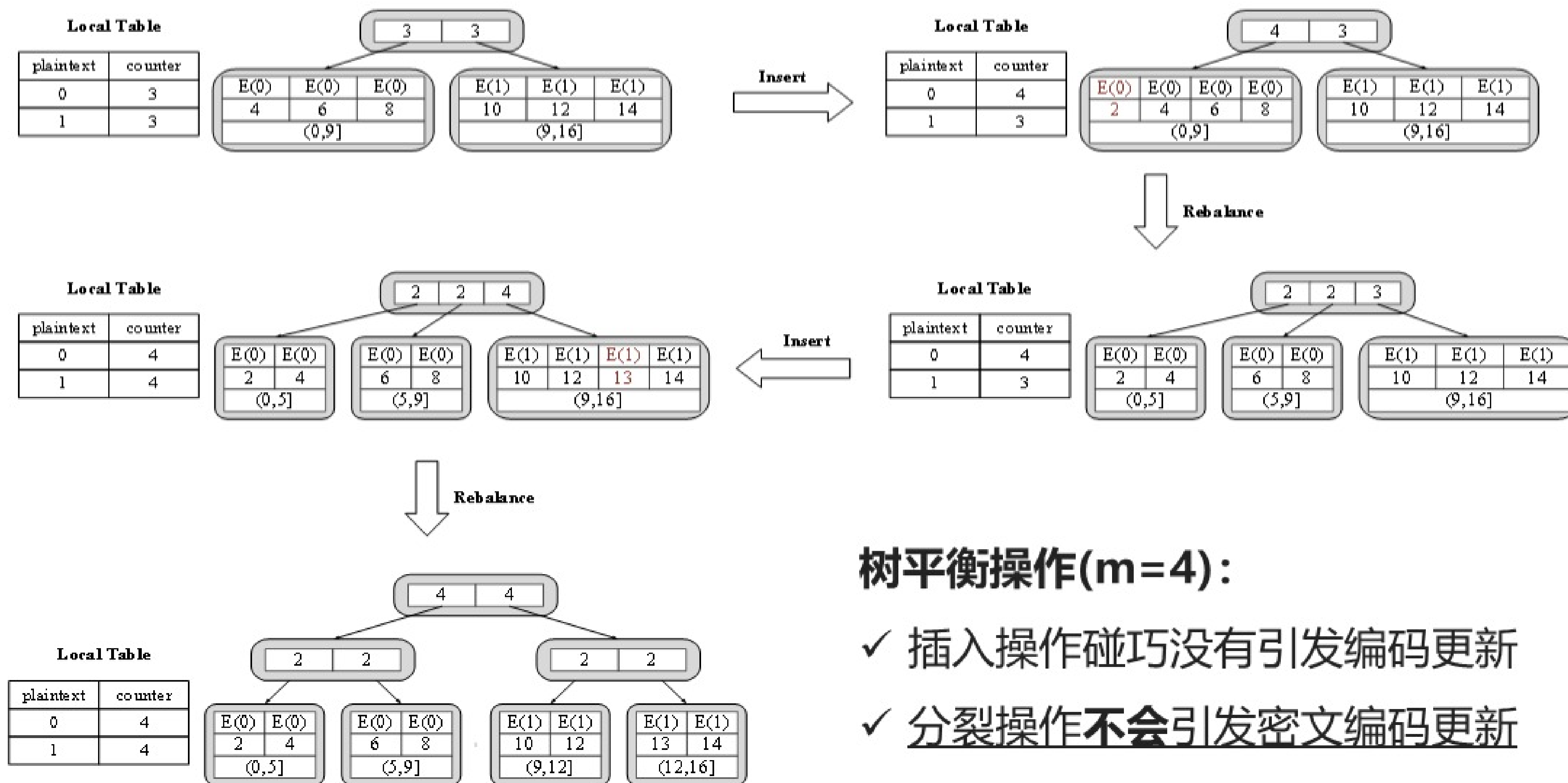
区域编码策略：

- 每个节点值区间为 $(a,b]$ ，默认新节点编码策略： $(L+R)/2$
- 节点内编码更新策略：区间 $(a,b]$ 、密文个数为 c ，更新后 $[1*(a+(b-a)/c), \dots, c*(a+(b-a)/c)]$



插入操作： 可能引发节点内数据密文编码更新，其他节点不发生更新

3.基于编码树的FH-OPE

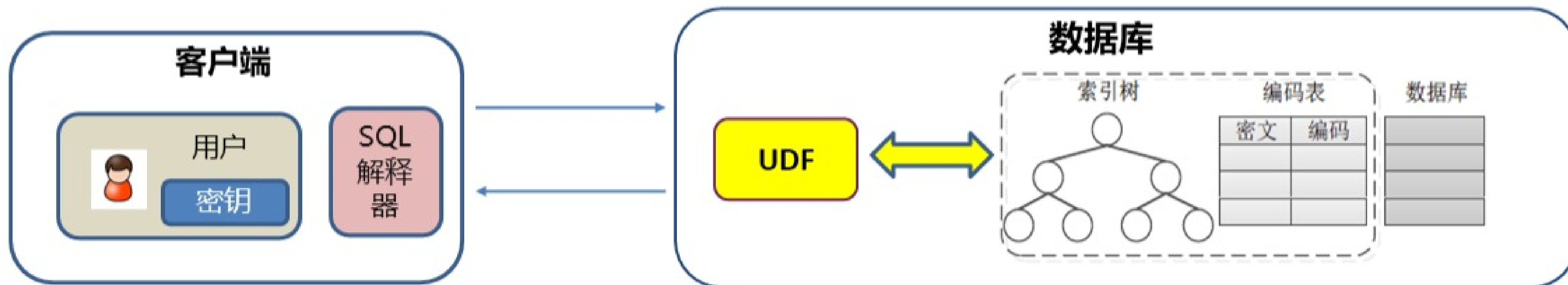


树平衡操作(m=4):

- ✓ 插入操作碰巧没有引发编码更新
- ✓ 分裂操作不会引发密文编码更新

4. 部署FH-OPE到数据库

客户端SQL改写；服务器端用户自定义函数UDF进行功能封装



示例：密文表T（密文ct，编码encoding）

• **插入**：call PRO_INSERT(pos, ct), 具体流程：

Create **procedure** PRO_INSERT(pos, ct)

BEGIN

Insert into T values(ct, FH_Insert(pos, ct));

如果满足条件，更新编码= FH_Update(ct)

END

• **查询**：范围(pt_{min}, pt_{max}) Select * from T where
FH_Search(pos_{min}) < 编码 and 编码 < FH_Search(pos_{max})

- *FHOPE_Insert(pos, ct)*。它以密文 *ct* 及其位置 *pos* 作为输入。然后，它调用块 *Insert(root, pos, ct)* 将 *ct* 插入到编码树上的 *pos* 中。如果插入操作不涉及任何编码更新，UDF 返回 *ct* 的编码 $cd = Insert(root, pos, ct)$ 。否则，UDF 使用 *M* 来记住所有更新的编码。
- *FHOPE_Update(ct)*。它以密文 *ct* 作为输入并返回 *cd*，它是 *M* 中 *ct* 的对应编码。
- *FHOPE_Update_Upper()/FHOPE_Update_Lower()*。它以 *M* 为单位返回编码的上限/下限。
- *FHOPE_Search(pos)*。它需要一个目标位置 *pos* 作为输入。然后调用块 $cd = GetCode(root, pos)$ 查询编码树上 *pos* 位置的密文。最后，UDF 返回编码 *cd*。

5.实验效果

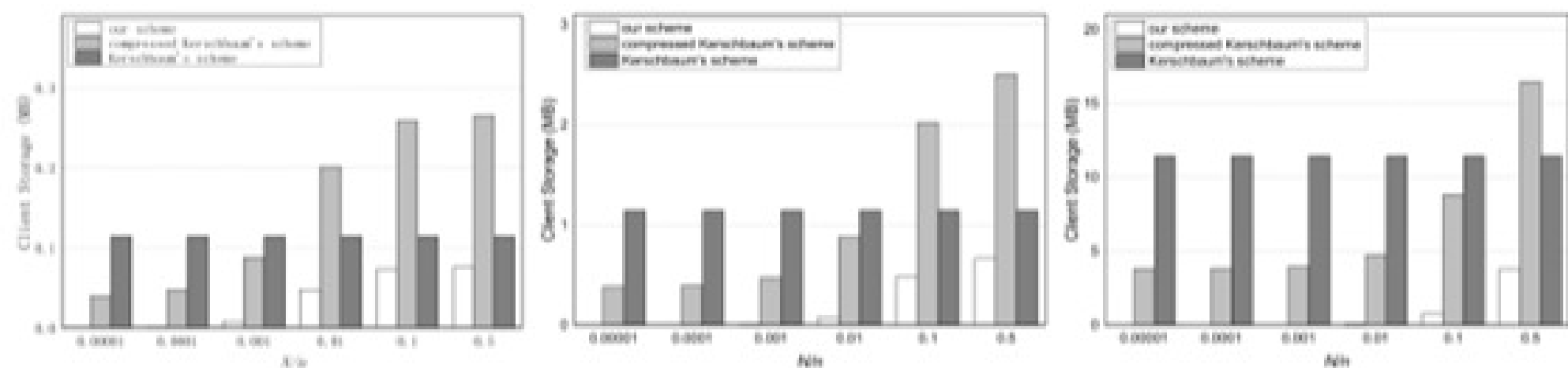


图 4.5 $n = 10^4$, Distribution 1

图 4.6 $n = 10^5$, Distribution 1

图 4.7 $n = 10^6$, Distribution 1

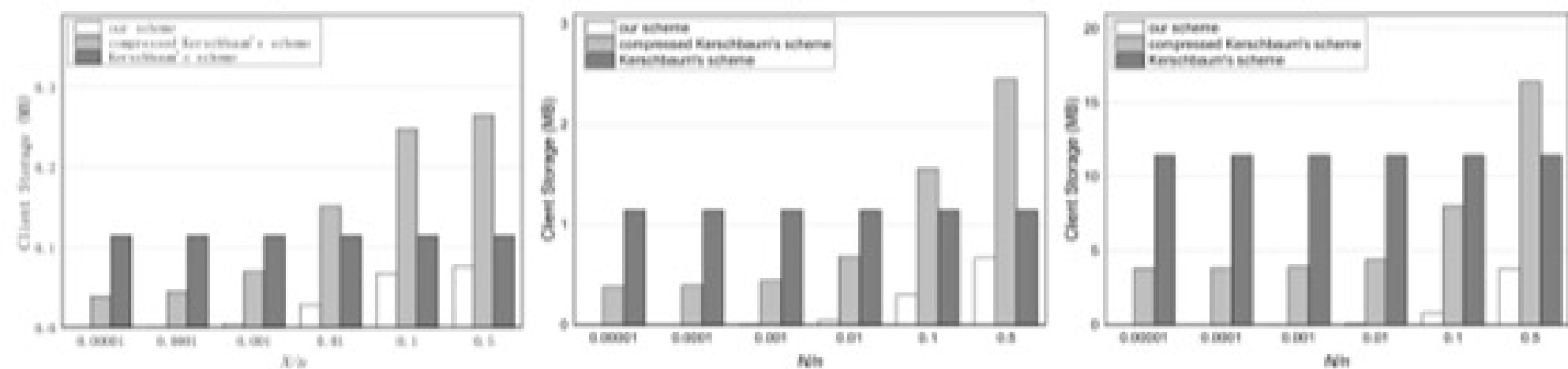


图 4.8 $n = 10^4$, Distribution 2

图 4.9 $n = 10^5$, Distribution 2

图 4.10 $n = 10^6$, Distribution 2

重新编码实验结果

实验结果
整体性能

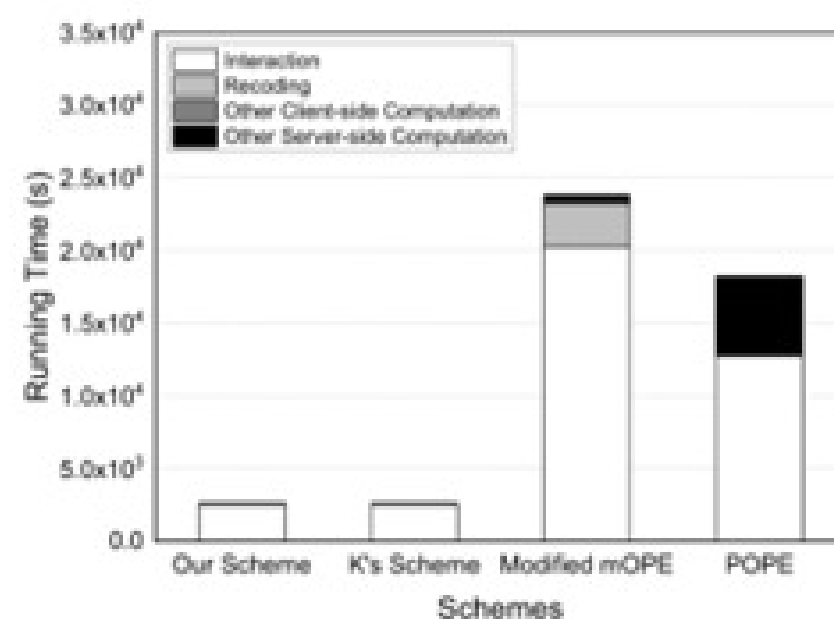


图 4.20 在 $-best$ 情况下的时间消耗

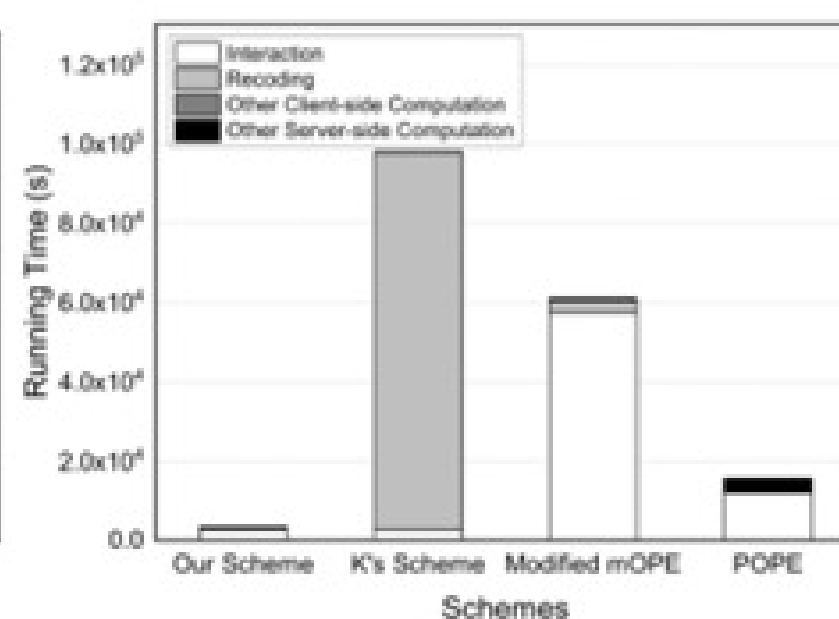


图 4.21 在 $-worst$ 情况下的时间消耗

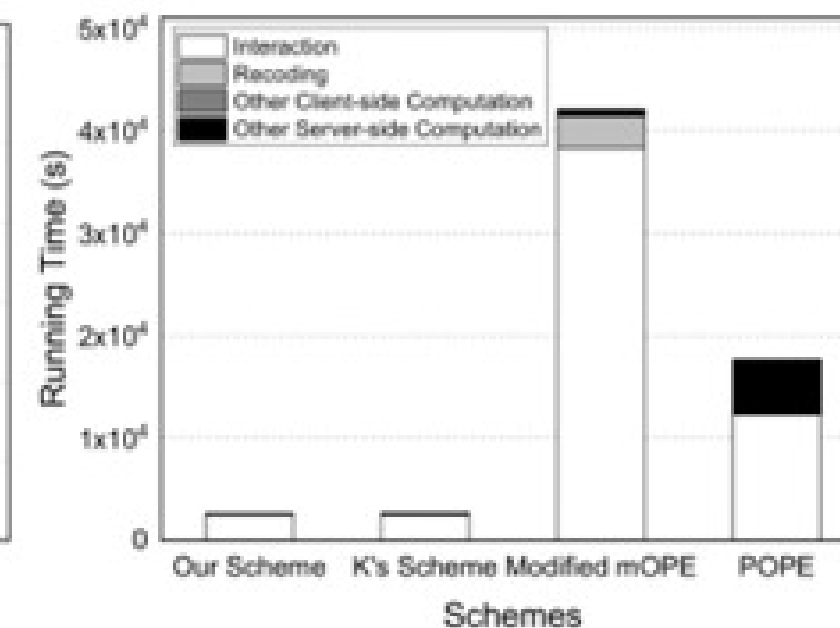


图 4.22 在 $-natural$ 情况下的时间消耗

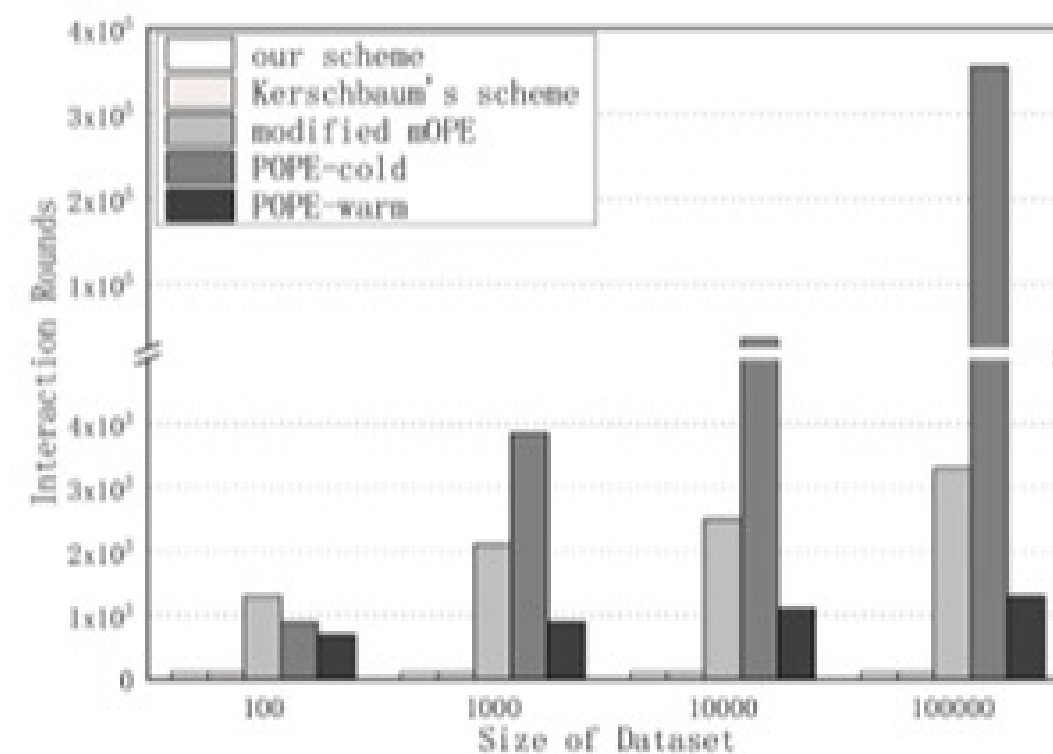


图 4.17 “Job Title” 的交互次数

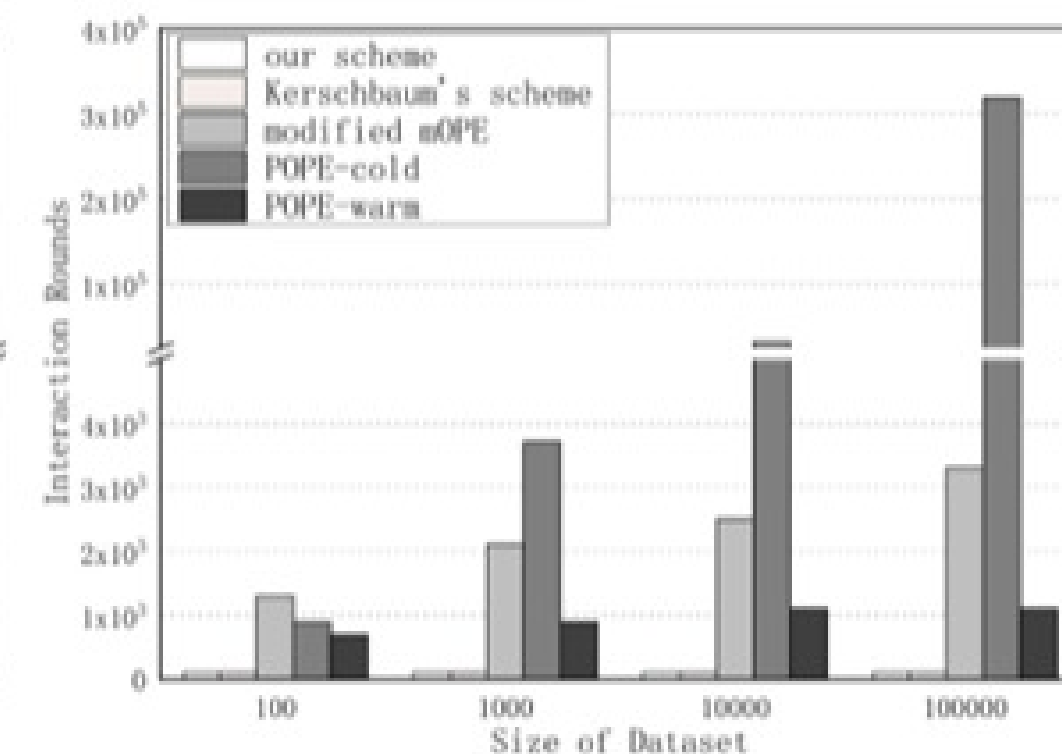


图 4.18 “Other Pay” 的交互次数

图 4.19 范围查询时的交互次数

交互次数实验结果

下列说法错误的是

- A 保留顺序加密可以让密文保留明文的顺序
- B 频率隐藏保序加密让相同明文的多个密文不相同
- C 达到IND-OCPA安全的保序加密方案就不存在其它安全风险
- D 频率隐藏保序加密可以抵御频率攻击

目 录

「01」

知识点五：保留顺序加密

「02」

知识点六：顺序揭示加密

「03」

知识点七：频率隐藏OPE

「04」

知识点八：FH-OPE实践

「05」

知识点九：密态数据库基础

「06」

知识点十：CryptDB数据库

1. 用户自定义函数

UDF (user-defined function) 用户自定义函数是数据库提供给开发者的扩展接口，使开发者不局限于数据库提供的功能，可以根据自身需求，利用UDF定制不同的功能函数。

以MySQL为例，使用UDF函数简单分为以下几个步骤：

- ✓ **编写C++程序文件**
- ✓ **编译生成动态链接库**
- ✓ **在数据库中创建函数**
- ✓ **调用函数**

1. 用户自定义函数

1. 编写C++程序文件

MYSQL为UDF提供了多种接口，其中至少需要实现主函数和初始化函数才可以运行。

(1) 主函数

该函数为用户自定义函数的主体部分，不同的返回类型具有不同的函数结构。

(2) 初始化函数

该函数在主体函数执行前由数据库自动调用，执行初始化工作，返回0表示正常。

```
bool <函数名>_init(UDF_INIT *initid, UDF_ARGS *args, char *message)
```

(3) 析构函数

该函数在主函数执行后调用，可在此释放资源。

```
void <函数名>_deinit(UDF_INIT *initid)
```

1. 用户自定义函数

1. 编写C++程序文件

数据类型： 编写UDF时主要涉及三种数据类型，其与C++数据类型的对应关系如下表

SQL数据类型	C++数据类型
INTEGER	long long
STRING	char *
REAL	double

函数原型：

`long long <函数名>(UDF_INIT *initid, UDF_ARGS *args, char *is_null, char *error)`

`char* <函数名>(UDF_INIT *initid, UDF_ARGS *args, char* result, char* length, char *is_null, char *error)`

`double <函数名>(UDF_INIT *initid, UDF_ARGS *args, char *is_null, char *error)`

1. 用户自定义函数

1. 编写C++程序文件

举例：两数相加的UDF示例。需要强调，C++中需要将函数封装于extern "C" 中。

```
#include "mysql/mysql.h"
extern "C"
{
    bool myadd_init(UDF_INIT *initid, UDF_ARGS *args, char *message);
    long long myadd(UDF_INIT *initid, UDF_ARGS *args, char *is_null, char *error);
}
bool myadd_init(UDF_INIT *initid, UDF_ARGS *args, char *message)
{
    return 0; }
long long myadd(UDF_INIT *initid, UDF_ARGS *args, char *is_null, char *error)
{
    int *x = (int *)args->args[0];
    int *y = (int *)args->args[1];
    return *x + *y; }
```

1. 用户自定义函数

2. 编译生成动态链接库

使用编译器编译的so文件，需要拷贝或链接到MySQL的plugin文件夹。一般而言，ubuntu下的默认地址为/usr/lib/mysql/plugin/。

以上步创建的myadd函数为例，假设代码文件为udf.cpp，编译输出文件名设定为udf.so。

```
g++ -shared -fPIC udf.cpp -o udf.so
```

```
[sudo] cp udf.so /usr/lib/mysql/plugin/
```


1. 用户自定义函数

3. 在数据库中创建函数

创建函数的命令为：

`CREATE FUNCTION` <函数名> `RETURNS` <返回值类型> `SONAME` <动态链接库文件名>

以myadd函数为例，创建函数命令为：

`CREATE FUNCTION` myadd `RETURNS INTEGER` `SONAME` 'udf.so';

需要注意的是，创建函数时需要确保已经进入某一数据库当中。使用如下命令可以创建数据库：`create database` <数据库名>

使用如下命令可以进入数据库：`use` <数据库名>

4. 调用函数

创建完成后，即可在SQL语句中调用该函数，例如我们可以直接通过select查询函数结果。

`select` myadd(1,2);

2. FH-OPE实现

详见教材

MYSQL

目 录

「01」

知识点五：保留顺序加密

「02」

知识点六：顺序揭示加密

「03」

知识点七：频率隐藏OPE

「04」

知识点八：FH-OPE实践

「05」

知识点九：密态数据库基础

「06」

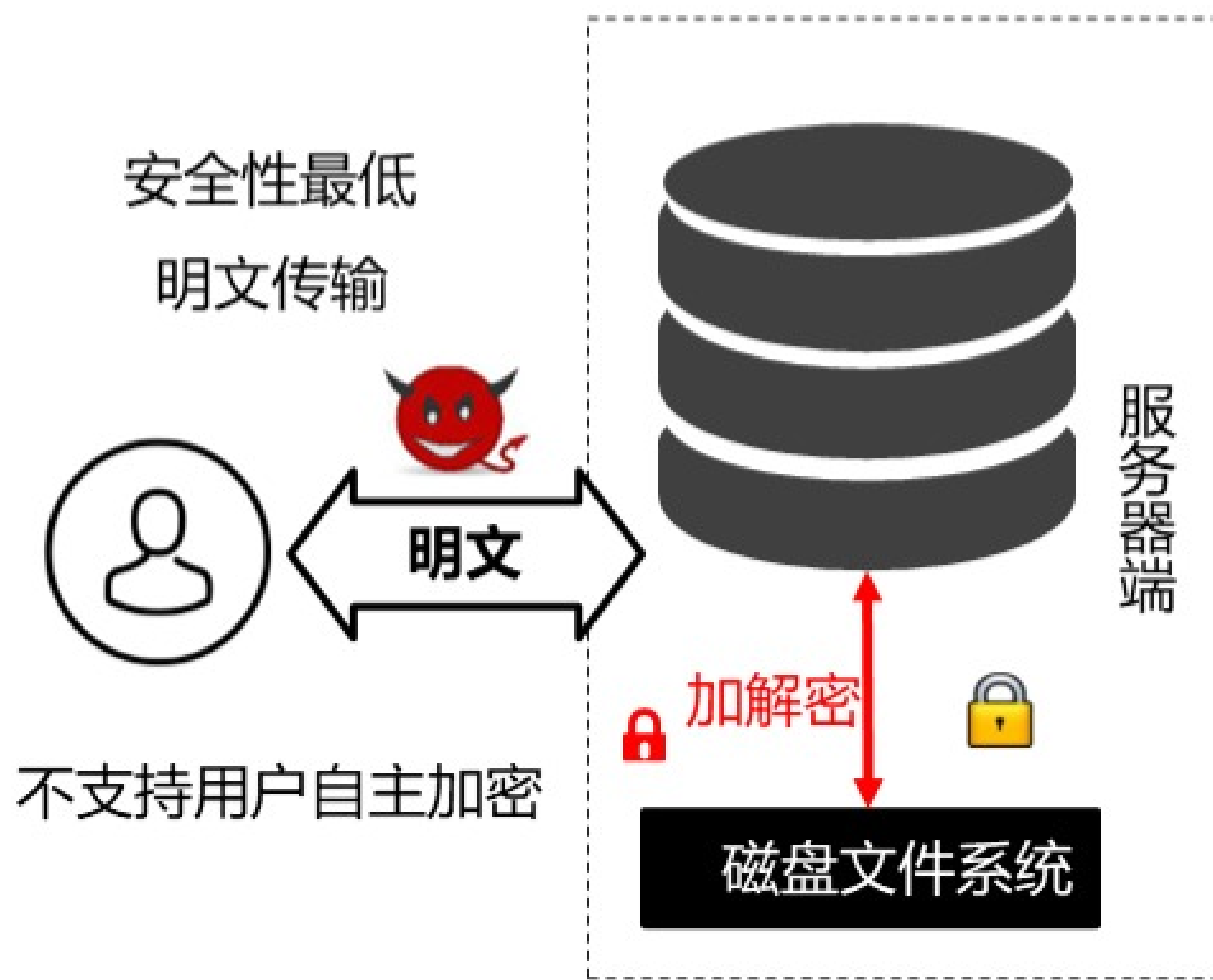
知识点十：CryptDB数据库

密态数据库的定义

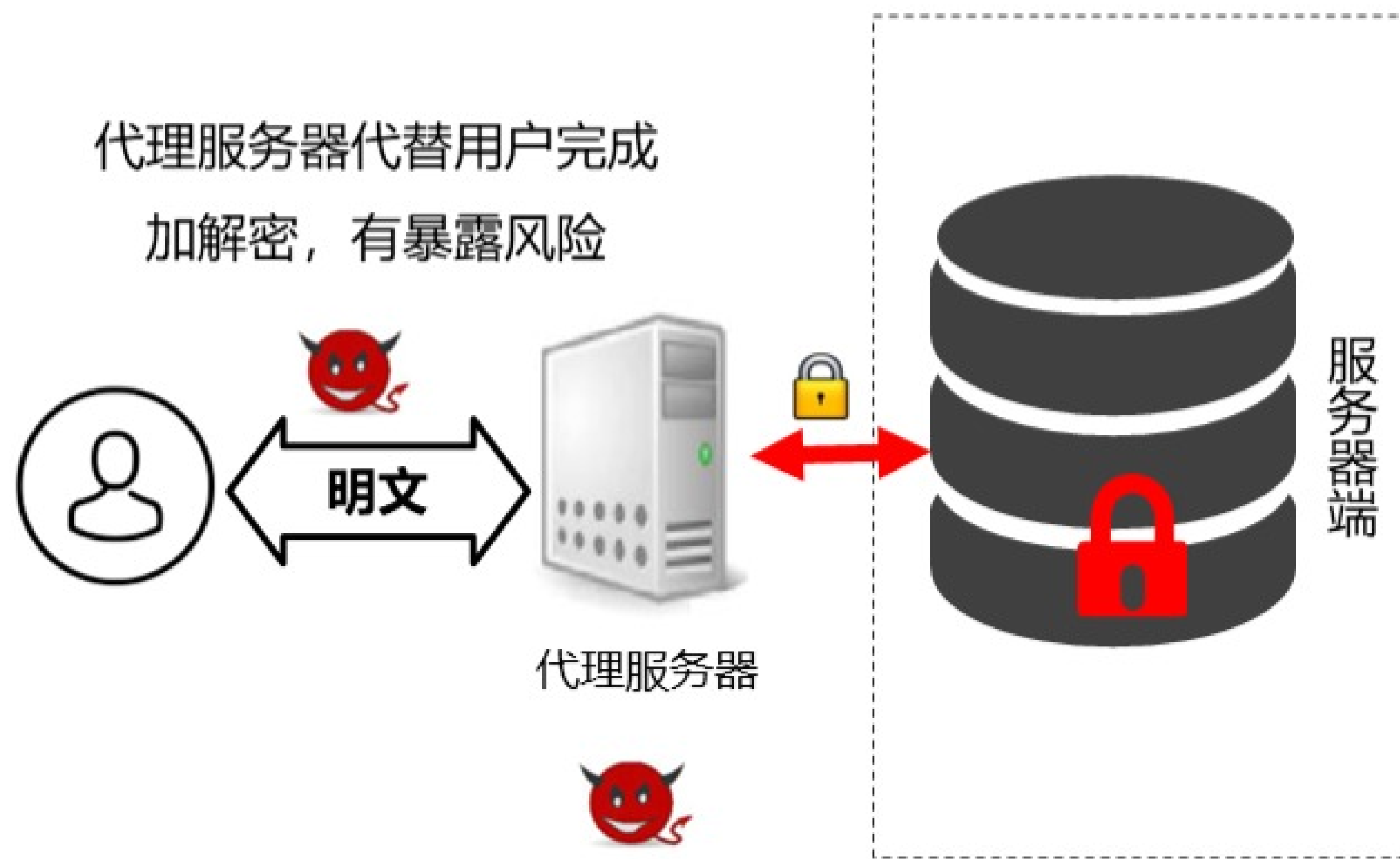
- **密态数据库是指存储和管理密态数据的数据库管理系统：**数据以加密形态存储在数据库中，其中数据存储、计算、检索、管理均在密文形态下完成，而与数据库管理相关的语法解析、事务等能力均集成传统数据库能力。
- **密态数据库是数据库系统与加密技术及数学算法深度结合的产物。**
- **密态数据库的核心任务是保护数据全生命周期的安全，并支持密态数据的检索和计算。**
- **分类：**基于纯密码学和基于可信执行环境的方案。

密态数据库的模型架构

主要有四类模式：**服务器磁盘加密**、**加密代理**、**可信执行环境**、**两端模式**



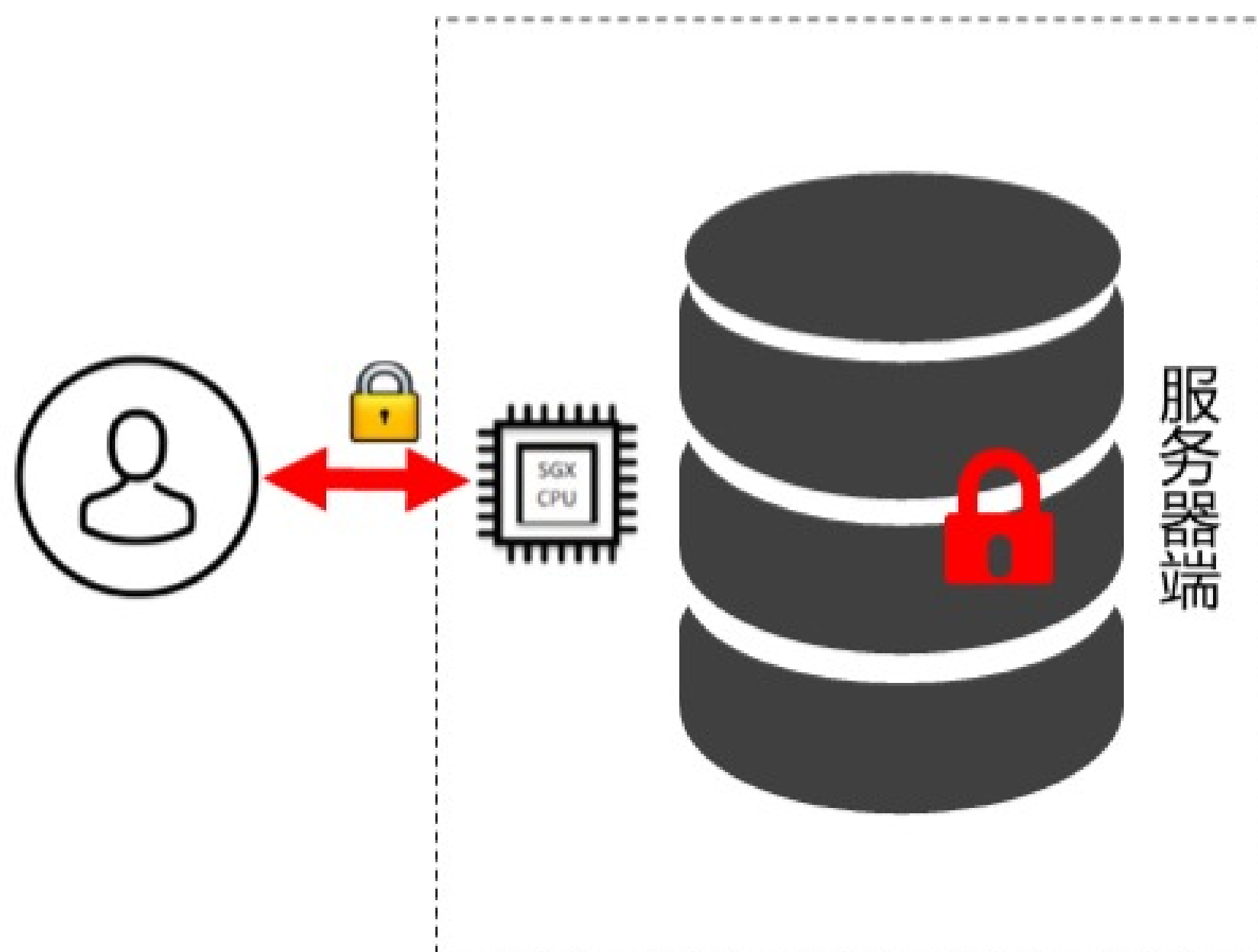
服务器端磁盘加密模式



加密代理模式

密态数据库的模型架构

主要有四类模式：服务器磁盘加密、加密代理、可信执行环境、两端模式



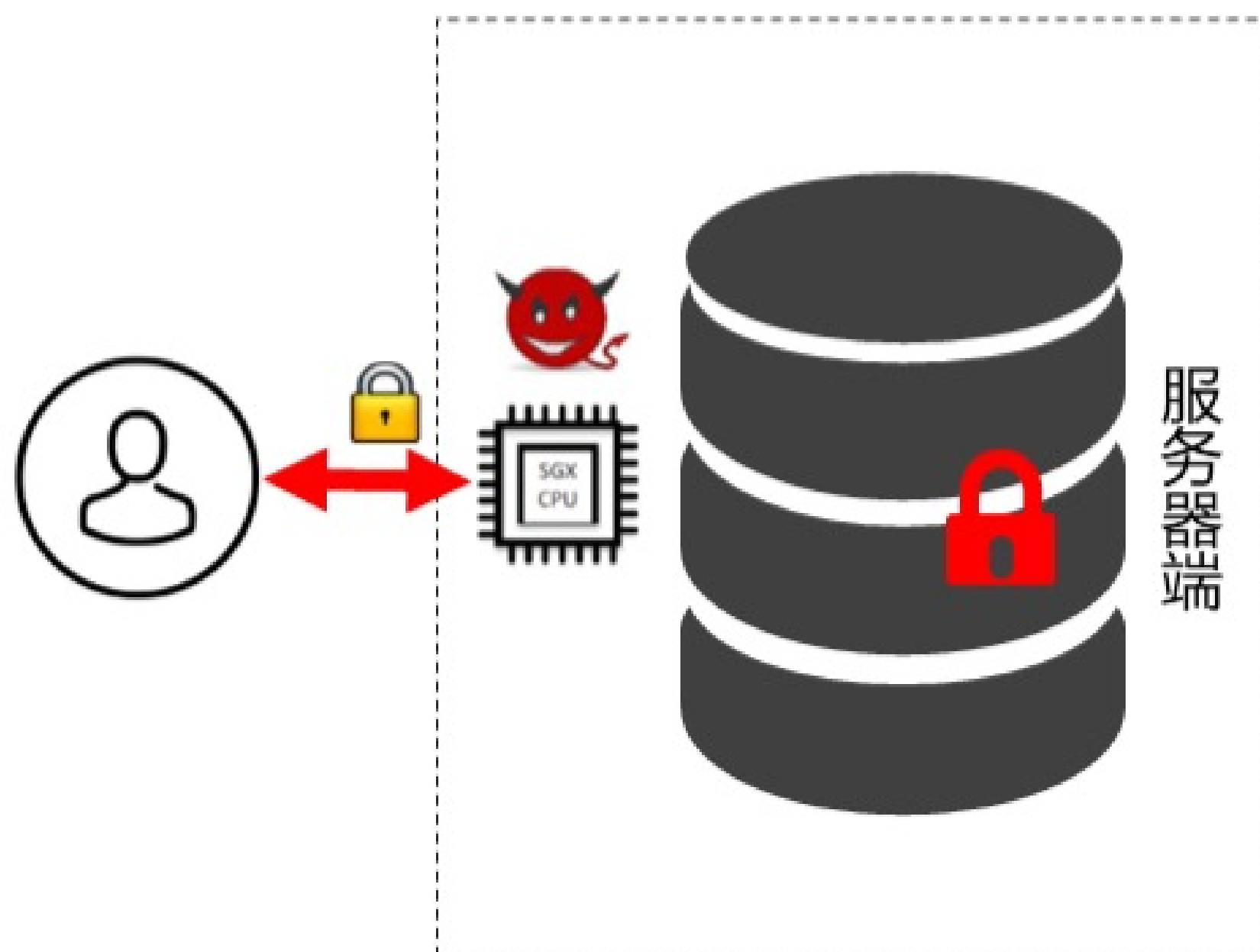
服务器可信执行环境模式

- 可信执行环境(TEE)可以在不信任的环境（例如云服务器）构建安全的应用程序，并具有较高的运行效率，已经成为当前商用化密态数据库的主研方向。
- 国内已有相关产品，华为的高斯数据库（ARM Trustzone、Intel SGX）、阿里的云数据库等。



密态数据库的模型架构

主要有四类模式：服务器磁盘加密、加密代理、可信执行环境、两端模式



服务器可信执行环境模式

性能约束：

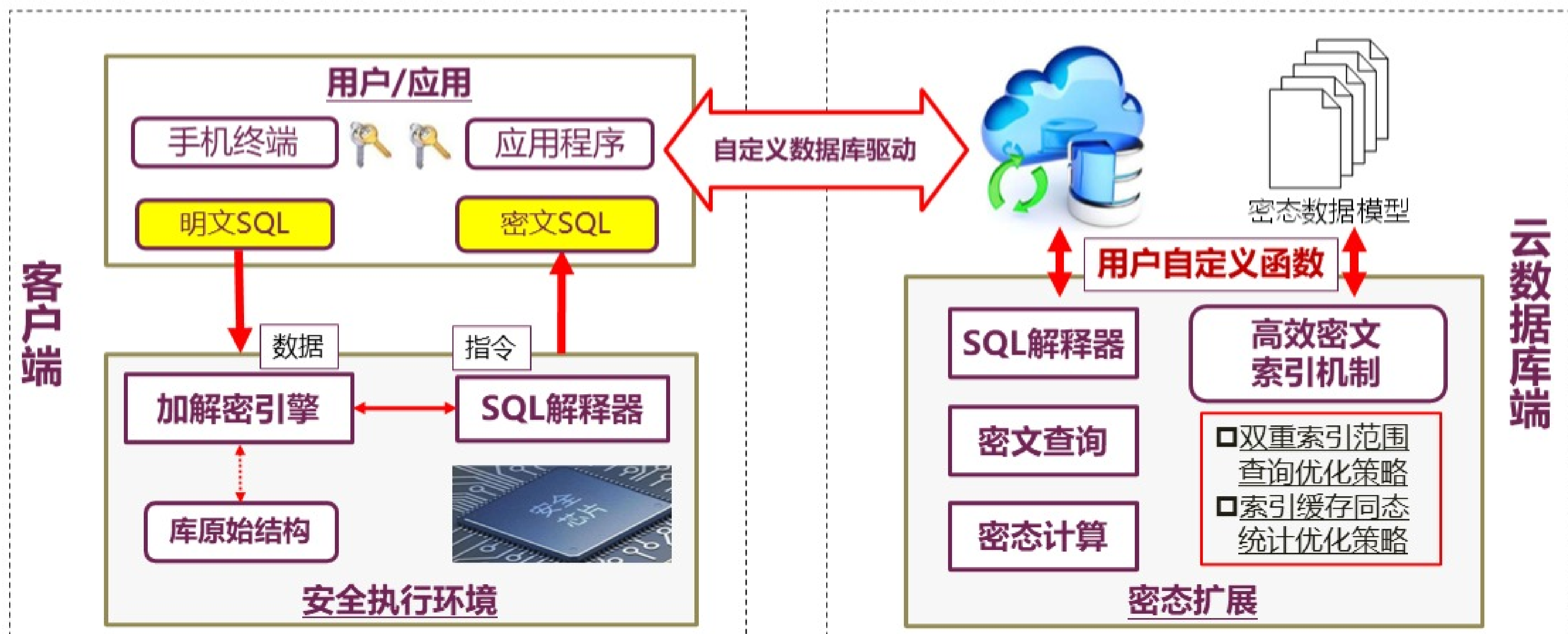
- 可信执行环境内存有限（SGX Enclave 128M）
- 状态切换开销大

安全约束：

- 可信执行环境是暴露面：攻击面大，容易遭受侧信道攻击、服务器同驻攻击等，安全性难以量化。
- 不保护访问行为模式：访问行为模式泄露已被证明可以用来完成敏感信息的推理。数据茫然（隐藏访问的数据）和程序茫然（隐藏执行分支）成为安全性基本要求，但是会带来额外的性能开销。

密态数据库的模型架构

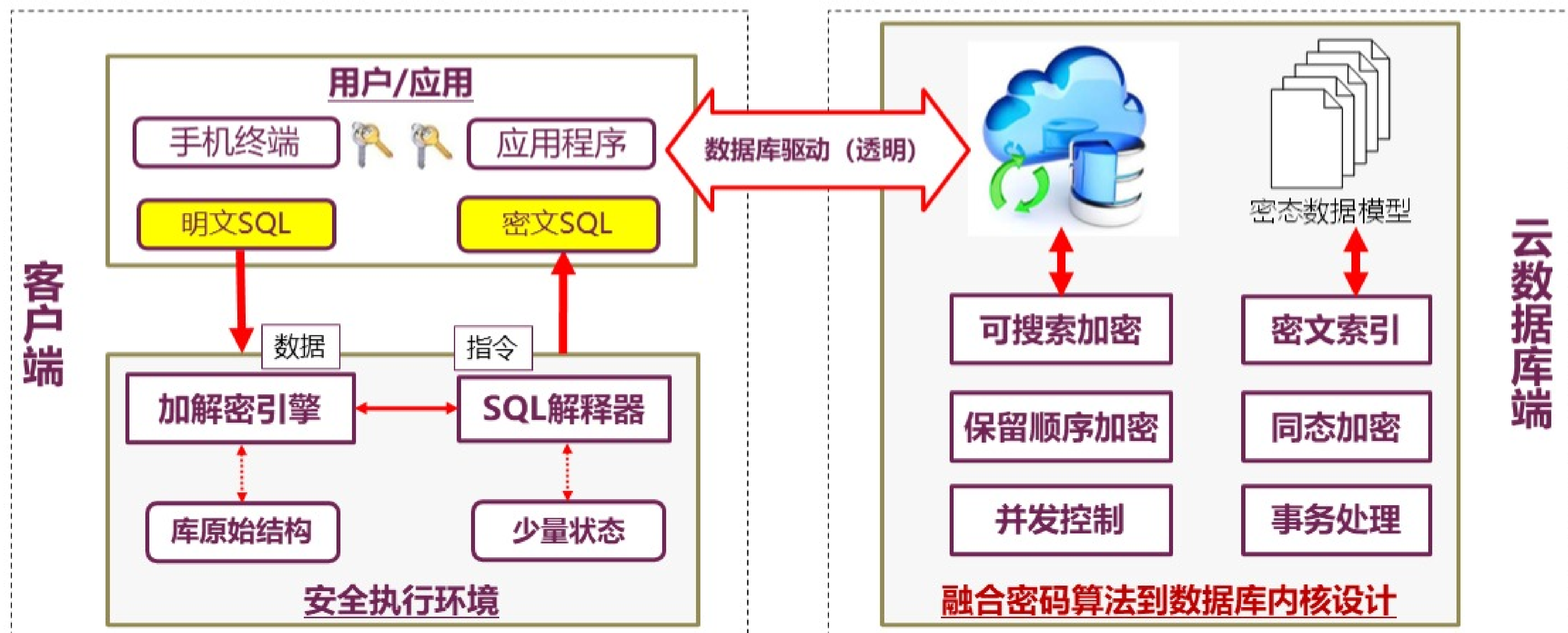
主要有四类模式：服务器磁盘加密、加密代理、可信执行环境、**两端模式**



两端模式也反应出密态数据库的理想工作模式：客户端有密钥，服务器端做计算

密态数据库的理想目标和工作模式

基于可证明安全的密码原语，构建安全性可衡量的算法级密态数据库



确定性加密 | 随机加密 | 同态加密 | 可搜索加密 | 保留顺序加密 | 顺序揭示加密

(1) 确定性加密算法 (DET)

确定性加密算法确定地为相同的明文生成相同的密文。

确定性加密算法会泄露相同明文的频率，难以抵御频率攻击，是密态数据库中最基本的加密机制。但是，确定性加密很实用，它允许服务器执行相等检查，这意味着它可以使用相等谓词、相等联接、分组依据、计数等功能。

(2) 随机加密算法 (RND)

随机加密是指将两个相等的值加密为不同的密文。

随机加密的一种有效构造是为每个相同的数据引入不同的随机数。比如，在分组密码CBC模式下使用随机初始化向量 (IV)。

随机加密不会泄露相同明文的频率，可以抵御离线攻击者的频率攻击。但是，随机加密会让相等谓词、相等联接、分组依据、计数等变得困难。

(3) 保序加密算法 (OPE)

OPE 的安全性要低于DET，OPE 的加密特点是明文数据项在加密后仍然保留着加密前的排序关系。如果对数据库中的某列数据进行OPE加密后，此列密文可以支持范围查询，大小比较等操作。

(4) 同态加密算法 (HOM)

HOM 是一种安全的概率加密方案，可以直接对密文进行计算。

为了支持求和运算 (sum、avg)，Paillier加法同态体制经常被选用。

(5) 可搜索加密算法 (SSE)

加密后仍能对密文文件、长文本密文数据进行关键词检索，允许不可信服务器无需解密就可以完成是否包含某关键词的判断。

密态数据库的安全目标



密态数据库的技术挑战

难题：随机化加密与功能(存储明文对应的密文)和性能(低存储、无交互)的冲突

描述：随机化加密要求一次一密，相同的明文具有不同的密文。因此，客户端需要掌握每个明文对应的密文或状态，导致要么客户端存储大量信息，要么需要和服务器交互来获得状态。**状态管理与低存储、无交互的冲突成为巨大挑战。**

随机化加密



客户端与服务器无交互

+

客户端低存储

现状：2016年陷入瓶颈，语义安全的数据库密文查询算法难有高效的解决办法

密态数据库的技术挑战

更高安全级别的茫然操作数据库成为未来自主创新的重要方向

1

查询模式

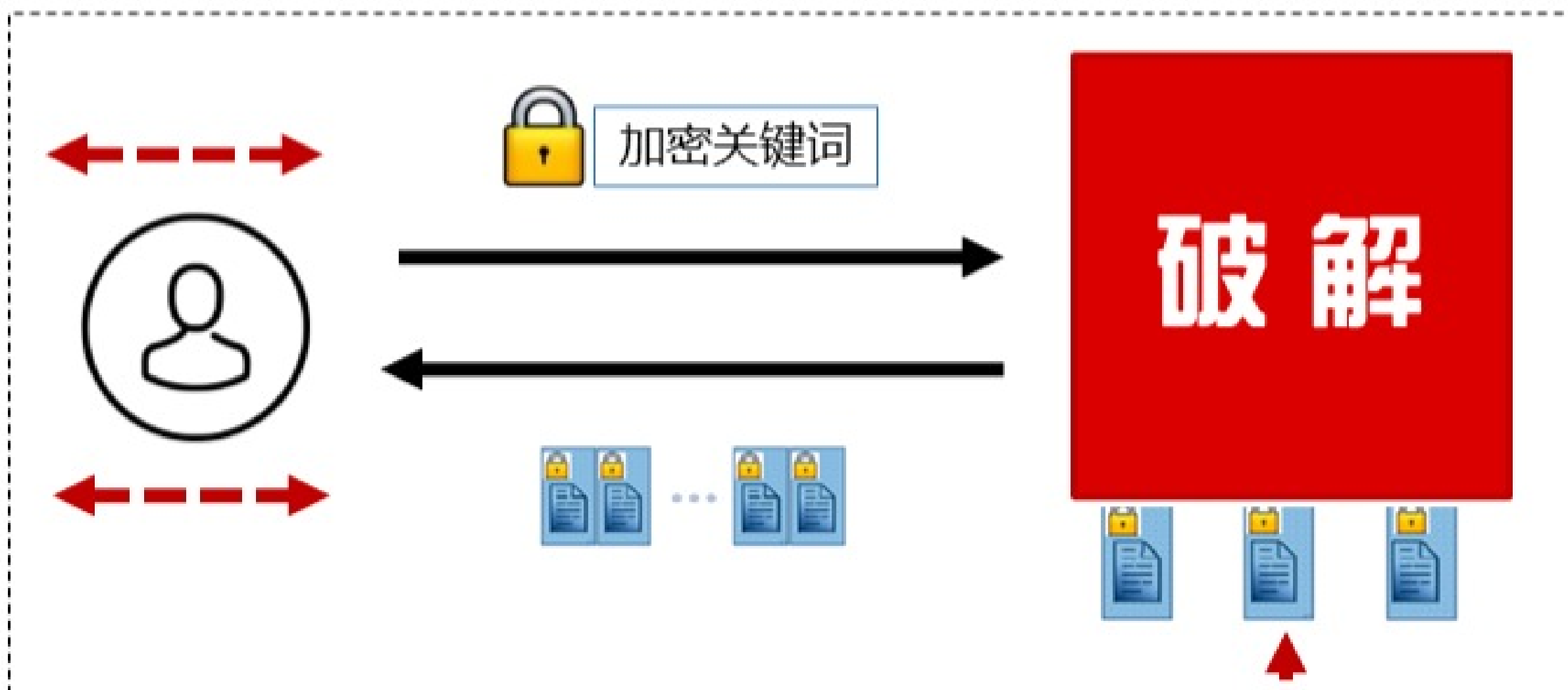
是否相同加密关键词的查询

2

大小模式

查询结果中包含的记录数量

这些模式泄露已被用来
完成对加密系统的破解



3

访问模式

哪些加密文件或者记录被访问

现状：探索阶段，基于茫然随机存储模型的解决方案性能都很低

目 录

「01」

知识点五：保留顺序加密

「02」

知识点六：顺序揭示加密

「03」

知识点七：频率隐藏OPE

「04」

知识点八：FH-OPE实践

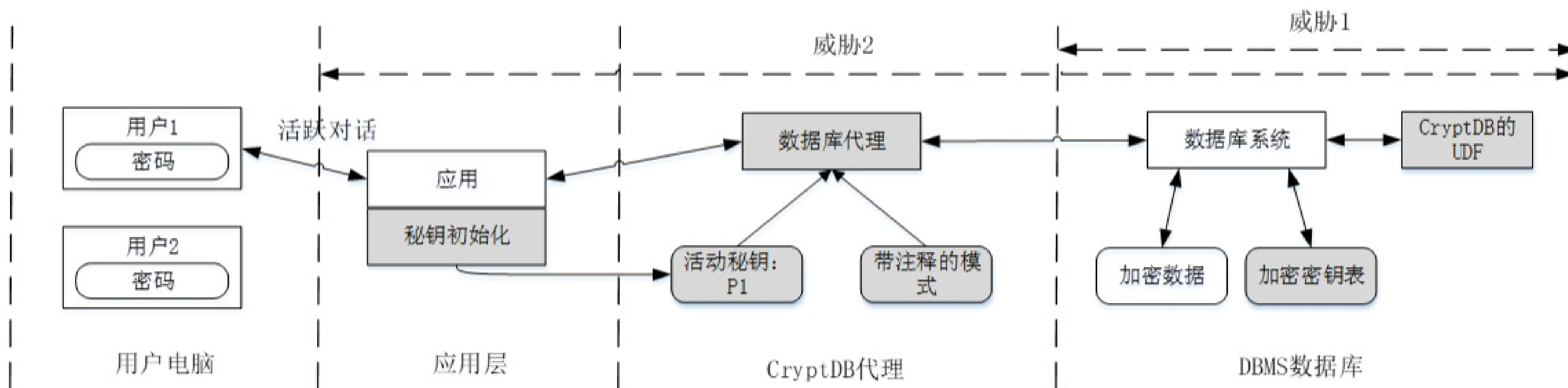
「05」

知识点九：密态数据库基础

「06」

知识点十：CryptDB数据库

1、系统架构

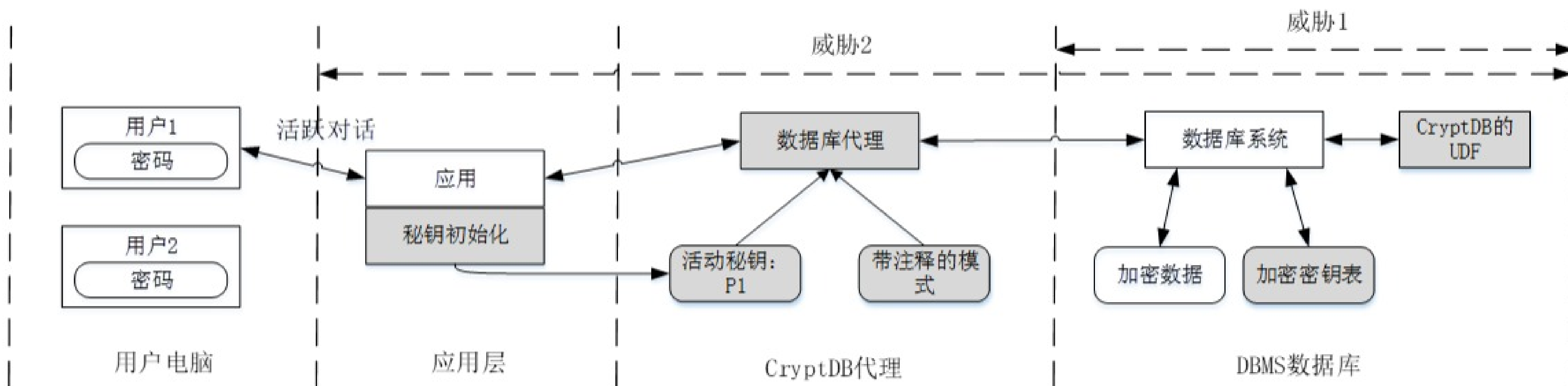


CryptDB的体系结构由两部分组成：**数据库代理**和**未修改的DBMS**。

CryptDB使用**用户定义函数UDF**在**DBMS**中执行加密操作。

矩形框和圆形框分别表示流程和数据。着色表示CryptDB添加的组件。虚线表示用户计算机、应用服务器、运行CryptDB数据库代理的服务器（通常与应用服务器相同）和DBMS服务器之间的分隔。

1、系统架构



CryptDB的工作原理是拦截数据库代理中的所有SQL查询，该代理将重写查询以在加密数据上执行（CryptDB假定所有查询都通过代理）。代理对所有数据进行加密和解密，并更改一些查询运算符，同时保留查询的语义。

代理：会存储主密钥MK、原始数据库表结构和对应的匿名后的表结构。

DBMS服务器：存在用户自定义函数，完成密文数据上的计算。

2、加密策略

CryptDB的加密策略分为六种，分别为随机加密算法RND、确定性加密算法DET、保序加密算法OPE、同态加密算法HOM、联结加密算法JOIN、搜索加密算法Search，每种加密算法支持不同类型的查询功能，具有不同的安全性。

连接加密算法（JOIN和OPE-JOIN）： CryptDB 中包括两种JOIN操作，分别是JOIN和OPE-JOIN。JOIN允许两列之间的等式连接，OPE-JOIN则通过顺序关系启用比较连接。在JOIN操作中，两个数据表为了完成 JOIN，应该对两列数据进行等值链接，JOIN还应该支持DET和SEARCH 所允许的所有操作，并且还允许服务器检测两列之间的重复值。

3、SQL解释

Employees

<i>ID</i>	<i>Name</i>
23	Alice

Table1

<i>C1-IV</i>	<i>C1-Eq</i>	<i>C1-Ord</i>	<i>C1-Add</i>	<i>C2-IV</i>	<i>C2-Eq</i>	<i>C2-Ord</i>	<i>C2-Search</i>
x27c3	x2b82	xcb94	xc2e4	x8a13	xd1e3	x7eb1	x29b0

(1) 表结构匿名化

表匿名化: Employees→Table1

字段匿名化:

ID→C1 Name→C2

根据用户对于字段加密的需求，分别产生对应的列

C1列要求同时支持相等查询、范围查询和加法运算

C2列要求通知支持相等查询、范围查询和关键词检索

原始表结构\用户

定义的加密策略存

储在代理服务器

3、SQL解释

Employees

<i>ID</i>	<i>Name</i>
23	Alice

Table1

<i>C1-IV</i>	<i>C1-Eq</i>	<i>C1-Ord</i>	<i>C1-Add</i>	<i>C2-IV</i>	<i>C2-Eq</i>	<i>C2-Ord</i>	<i>C2-Search</i>
x27c3	x2b82	xcb94	xc2e4	x8a13	xd1e3	x7eb1	x29b0

(2) SQL解释

Insert into Employees(ID, Name) values(24, 'Bob');

解释为

Insert into Table1(C1-IV,C1-Eq,C1-Ord,C1-Add,...)

values(RND(24), DET(24),OPE(24),HOM(24),...);

Select * from Employees where ID>24

解释为

Select * from Table1 where C1-Ord > OPE(24)

解释过程：代理

RND等是UDF:

服务器端

4、洋葱加密

加密策略

CryptDB的加密策略分为六种，分别为随机加密算法RND、确定性加密算法DET、保序加密算法OPE、同态加密算法HOM、联结加密算法JOIN、搜索加密算法Search，每种加密算法支持不同类型的查询功能，具有不同的安全性。

连接加密算法（JOIN和OPE-JOIN）： CryptDB 中包括两种JOIN操作，分别是JOIN和OPE-JOIN。JOIN允许两列之间的等式连接，OPE-JOIN则通过顺序关系启用比较连接。在JOIN操作中，两个数据表为了完成 JOIN，应该对两列数据进行等值链接，JOIN还应该支持DET和SEARCH 所允许的所有操作，并且还允许服务器检测两列之间的重复值。

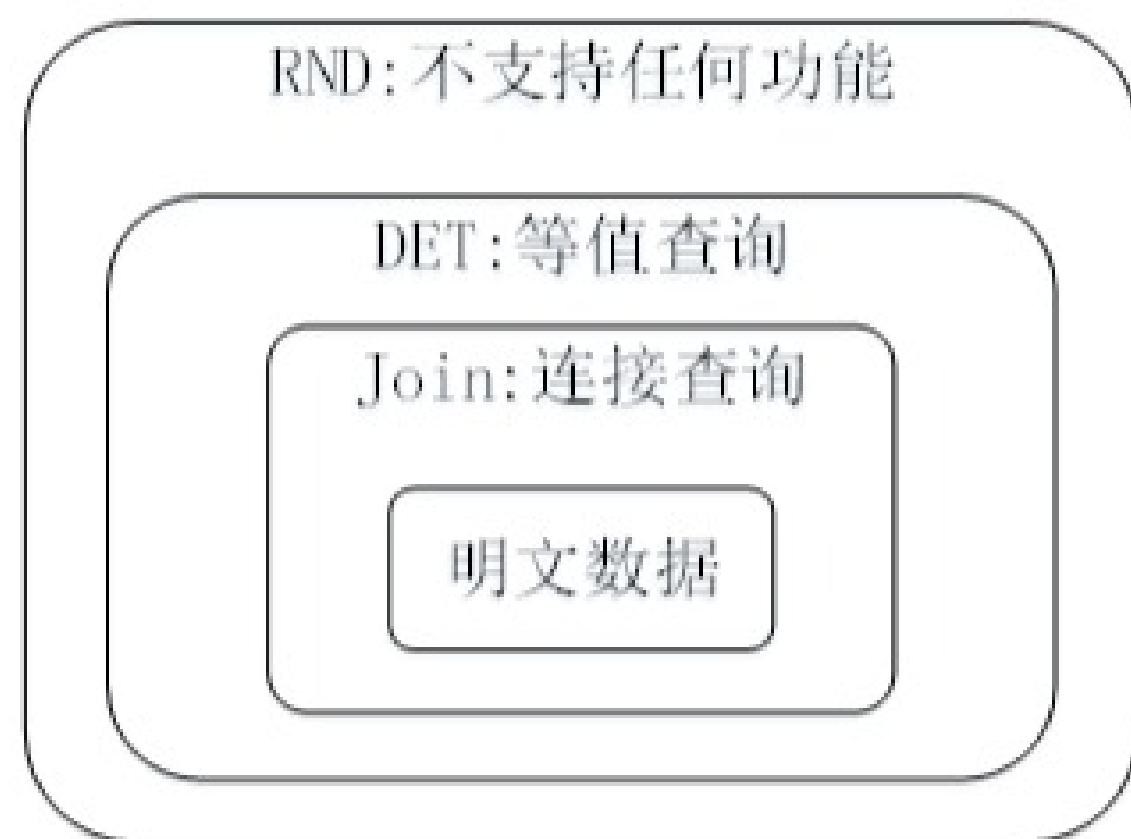
4、洋葱加密

洋葱加密

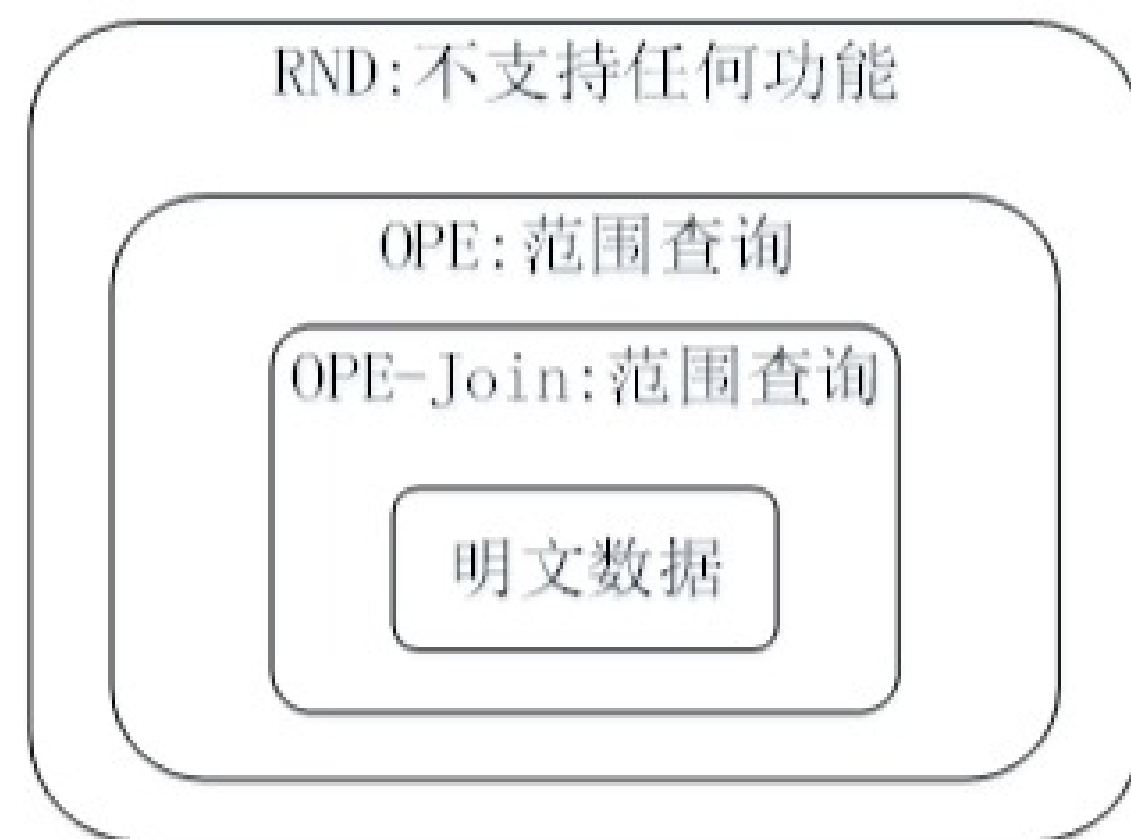
CryptDB采用了六种加密方案：安全性上， $OPE < DET < RND$ 。

为了取得更高的安全性和支持更多的操作，比如，RND很难支持链接查询，而DET很容易支持链接查询。CryptDB设计了洋葱加密，使得默认工作在最安全层，一旦需要不能支持的操作的时候，可以剥掉洋葱，加入下一层，进而牺牲安全性，换来更多操作的支持。**洋葱由内到外，安全性越来越强，通常内部的洋葱提供功能性，最外层的洋葱则提供了最高级的安全性。**

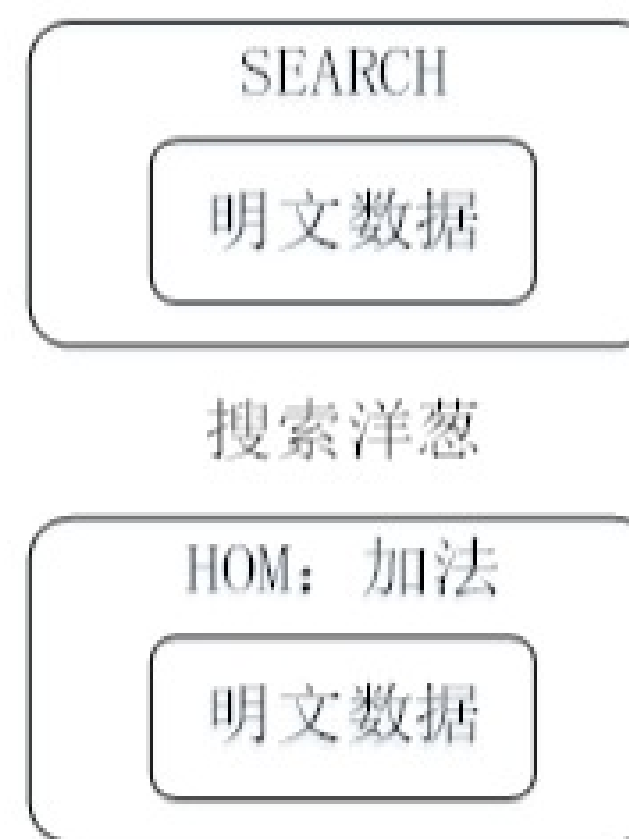
4、洋葱加密



相等查询洋葱



范围查询洋葱

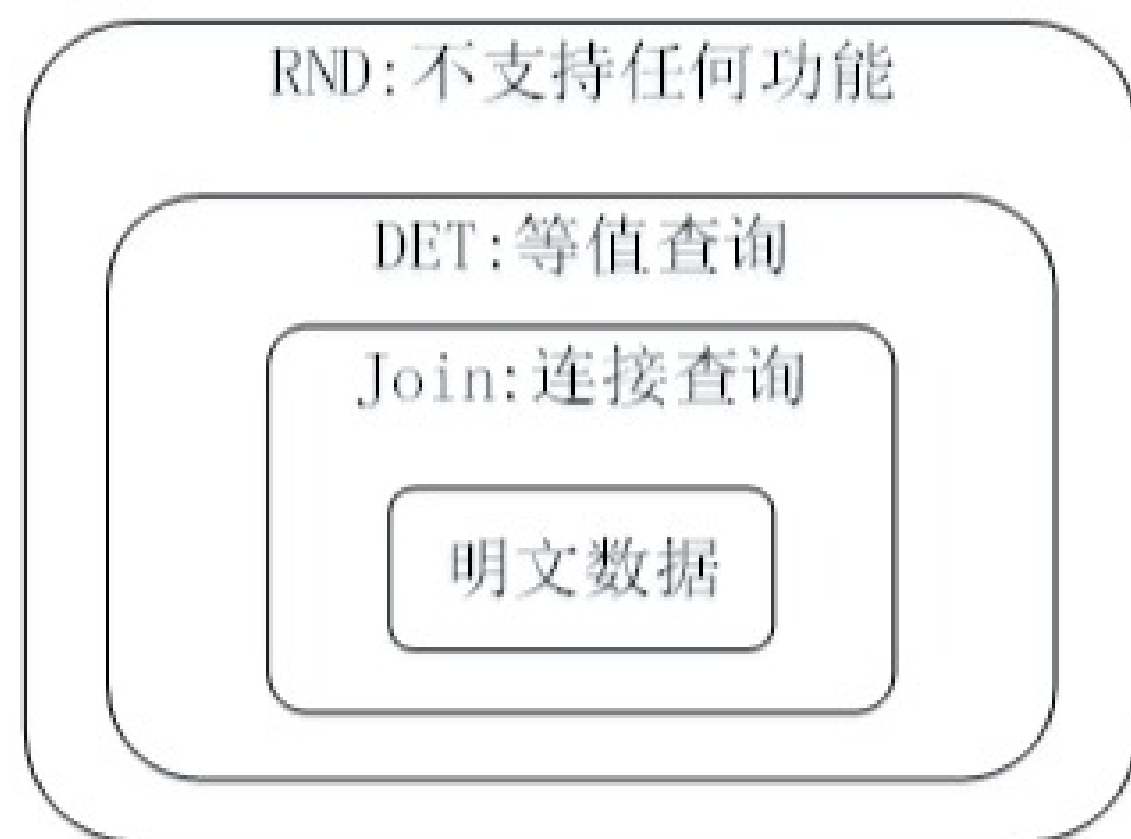


加法洋葱

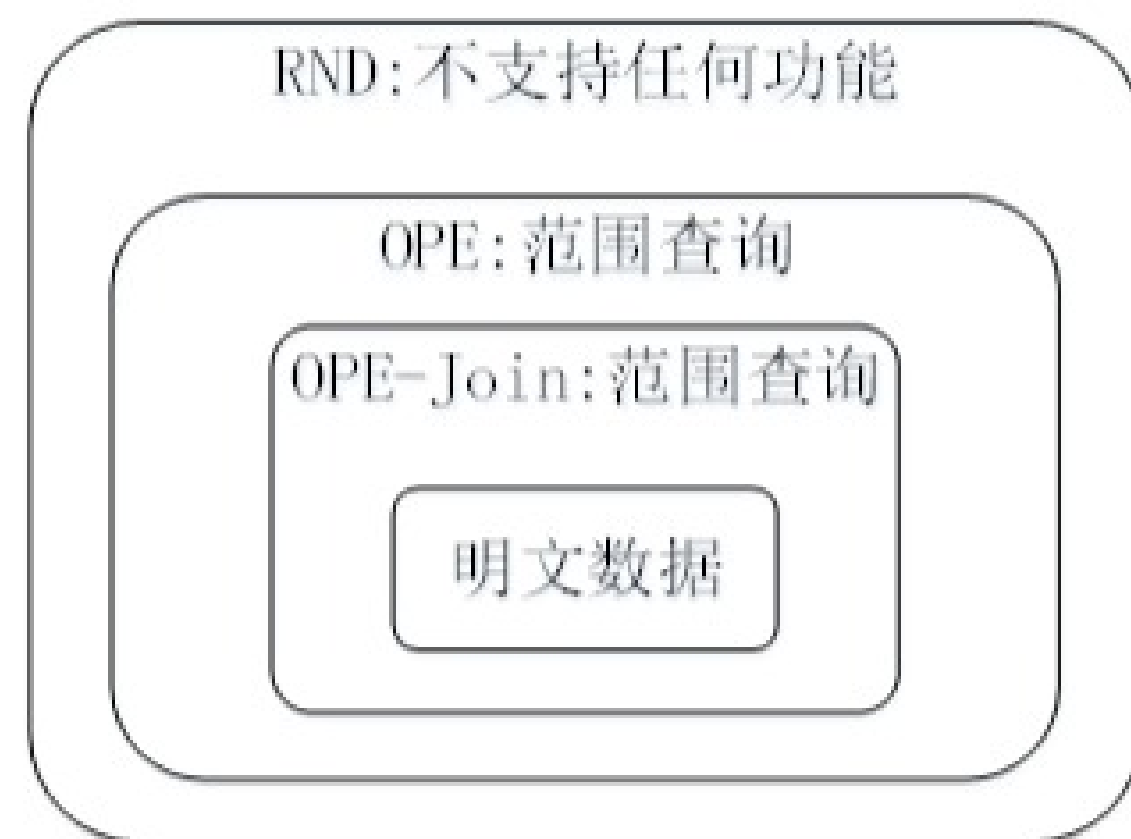
CryptDB设计了四种加密洋葱，等值洋葱、范围查询洋葱、加法洋葱、搜索洋葱，分别支持等值查询、范围查询、同态加与模糊搜索功能。

- 洋葱外层一般用安全性较高的RND和 HOM能够保证安全性；
- 内层的DET和JOIN安全性较弱，但能提供功能性的计算。

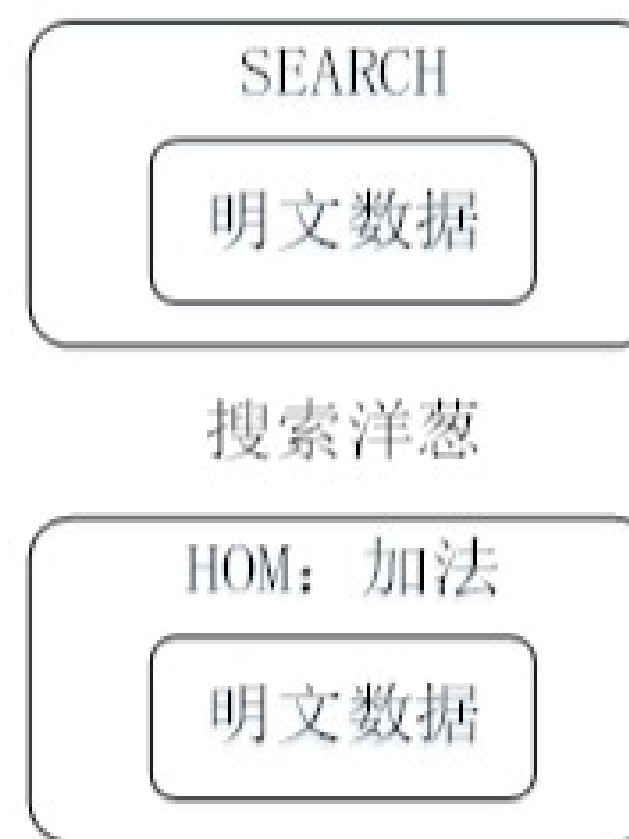
4、洋葱加密



相等查询洋葱



范围查询洋葱



搜索洋葱

加法洋葱

(2) 动态调整策略

每个洋葱一开始都使用最安全的加密方案进行加密。

当代理从应用程序接收SQL查询时，它决定是否需要删除加密层。

CryptDB使用在DBMS服务器上运行的UDF实现洋葱层解密。

4、洋葱加密

Employees

<i>ID</i>	<i>Name</i>
23	Alice

Table1

<i>C1-IV</i>	<i>C1-Eq</i>	<i>C1-Ord</i>	<i>C1-Add</i>	<i>C2-IV</i>	<i>C2-Eq</i>	<i>C2-Ord</i>	<i>C2-Search</i>
x27c3	x2b82	xcb94	xc2e4	x8a13	xd1e3	x7eb1	x29b0

例如，在上图中，要将表1中第2列的范围查询洋葱解密到层OPE，代理使用 DECRYPT_RND UDF向服务器发出以下查询：

UPDATE Table1 SET C2-Ord = DECRYPT_RND(K, C2-Ord, C2-IV)

其中K是计算的适当密钥。同时，代理更新自己的内部状态，以记住表1中的C2列Ord 现在位于DBMS中的OPE层。

4、洋葱加密

缺点：

因为要剥掉洋葱，因此，剥掉洋葱后就会降低安全性。

此外，为了安全性，必须重新将洋葱加上去来保证安全性，这又带来了极大的性能损失。

5、索引

DBMS以与明文相同的方式为加密数据建立索引。

当前，如果应用程序请求列上的索引，则代理将要求DBMS服务器在该列的DET、JOIN、OPE或OPE-JOIN洋葱层（如果已公开）上构建索引，但不针对RND、HOM或搜索。

可以研究更有效的索引选择算法。

6、连接查询

支持两种类型的连接：**等值连接和范围连接（涉及顺序检查）**。

- 要对两个加密列执行等联接，**应使用相同的密钥对这些列进行加密，以便服务器可以看到两个列之间的匹配值。**
- **挑战：**为了更好的隐私，DBMS服务器不应该能够连接应用程序未请求连接的列，因此**从未连接的列不应该使用相同的密钥加密。**
- 为了解决这个问题，CryptDB引入了一个新的**加密原语JOIN-ADJ（可调连接）**，它允许DBMS服务器在运行时调整每列的密钥。

谢 谢

